

DeweyLib

The Complete Lab Walkthrough

From an Empty Folder to a Working REST API

A step-by-step guide written for someone who has never written a line of C# before.
Every significant step is explained — not just what to type, but why, and exactly which file it goes in.

Second Edition — revised with file-placement guidance and a documented log of every change made while building the reference solution.

How to Use This Document

This guide walks you through building DeweyLib, a small library management system, completely from scratch. By the time you reach the last page, you will have a working console application and a working REST API — built entirely by you, one small piece at a time.

The guide is organized into 12 Milestones, grouped into three tiers:

- Milestones 1–4 (Beginner) — console basics, methods, and your first class
- Milestones 5–8 (Intermediate) — inheritance, generics, file storage, and LINQ
- Milestones 9–12 (Advanced) — events, async programming, reflection, and a REST API

Each milestone follows the same pattern:

1. A plain-English explanation of the goal — what you're building and why it matters, before any code appears.
2. A file-placement callout — telling you exactly which file the upcoming code belongs in, and whether it's a new file you need to create.
3. Step-by-step instructions — broken into small, checkable pieces.
4. The complete code for that step — so you can compare your work against a known-good answer.
5. A line-by-line explanation of anything unfamiliar — we never assume you already know a keyword or symbol.
6. A checkpoint — a clear, observable way to confirm the step worked before moving on.

 **Tip:** You do not need to type every line from memory. Read the explanation first, try writing the code yourself, and only look at the provided solution if you get stuck or want to check your work.

 **New in this edition:** Every code block is preceded by a  **FILE** box telling you exactly which file to open (or create) and roughly where in that file the new code belongs. If you're ever staring at a code sample wondering “...but where does this go?” — look one box up. Boxes marked  **NEW FILE** mean exactly that: create a brand-new, empty file with that name before pasting anything in.

 **Before you start:** Install the .NET 10 SDK from dotnet.microsoft.com if you haven't already, and install VS Code with the C# Dev Kit extension (or Visual Studio 2026 Community). Open a terminal and run `dotnet --version` to confirm you see 10.x.x before continuing.

What You're Building

DeweyLib is a library management system — the same kind of software a small library might use to track its books, DVDs, and magazines. Real libraries need to answer questions like:

- What items do we have, and which ones are currently checked out?
- When is a checked-out item due back, and is it overdue?
- If someone wants an item that's unavailable, can we notify them when it's returned?
- Can other systems — a website, a mobile app — ask our system questions over the internet?

By the end of this guide, your program will be able to answer all of these questions. You'll build it in stages, and at every stage you'll have something that actually runs — there's no point where the program is broken and unusable while you wait for a future milestone to fix it.

The Complete Map

Here is everything you will build, in order. Don't worry about understanding every term yet — each one is explained fully when you reach it.

M1 Console Menu & Input — Print text to the screen and read what the user types.

M2 Validation & Loops — Make the menu repeat forever and reject bad input safely.

M3 Methods — Break a long program into small, named, reusable pieces.

M4 Your First Class — Replace fragile data with a real Book object that protects its own rules.

M5 Inheritance — Let Book, Dvd, and Magazine share common behavior automatically.

M6 Generics & Interfaces — Build one storage container that works for any type of data.

M7 Exceptions & File Storage — Handle errors properly and save data so it survives a restart.

M8 LINQ Reports — Query and summarize your data in a single readable line of code.

M9 Events — Let one part of the program announce something happened, without knowing who's listening.

M10 Async Programming — Do slow operations (like network calls) without freezing the program.

M11 Reflection — Let new features register themselves automatically, with no extra wiring.

M12 Web API — Expose your library over the internet so other programs can use it.

 **The big idea:** Notice that nothing in this list is inherently "hard." Each milestone adds exactly one new idea on top of things you already know. By the time you reach the Web API at the very end, it will feel like a small step — because everything underneath it is already familiar.

How Your Project Will Be Organized

One more thing that trips up beginners: in Milestones 1 through 11, everything lives in ONE project, in a folder you'll create in Milestone 1 called DeweyLib. As the milestones progress, you'll create new .cs files inside that SAME folder — you are never creating new projects until Milestone 12. Here's the complete file list you'll have by the end of Milestone 11, so you always know what should already exist:

- `Program.cs` — created in Milestone 1; holds your menu and all top-level logic for the whole guide.
- `LibraryItem.cs` — created in Milestone 5; the Book/Dvd/Magazine hierarchy.
- `Repository.cs` — created in Milestone 6; the generic storage interface and implementation.
- `Exceptions.cs` — created in Milestone 7; your two custom exception types.
- `CheckoutService.cs` — created in Milestone 7; gains an event in Milestone 9.
- `LibraryStorage.cs` — created in Milestone 7; JSON save/load.
- `Reports.cs` — created in Milestone 8; rewritten in Milestone 11 to use attributes.
- `Events.cs` — created in Milestone 9; event-args classes and `HoldWatcher`.
- `AsyncClients.cs` — created in Milestone 10; the simulated network client and async feed.
- `ReportAttribute.cs` — created in Milestone 11; the `[Report]` attribute.
- `ReportRunner.cs` — created in Milestone 11; the reflection-based discovery engine.



Why so many files? You could put everything in `Program.cs` and the program would work identically — C# doesn't care how you split files. We split things up because it's how real C# projects are organized: one class (or a few closely related ones) per file, named after what's inside. Getting in this habit now will make any future C# codebase you open feel familiar instead of overwhelming.

Table of Contents

Right-click anywhere in the table below and choose “Update Field” if your Word processor doesn’t render page numbers automatically when you first open this document.

PART I

The Beginner Tier

Milestones 1–4 — Console basics, methods, and your first class

Milestone 1: Console Menu & Input

What You're Building

Every program needs a way to talk to the person using it. The simplest way to do that in C# is through the console — the black (or white) text window where you type commands. In this milestone, you'll write a program that displays a welcome message, asks for the user's name, greets them by name, and shows a menu of options.

This sounds simple, and it is — but it introduces three ideas you will use in literally every program you ever write: printing text, reading input, and combining text with data (string interpolation).

Step 1.1 — Create the Project

Before writing any code, you need a project for it to live in. Open a terminal (in VS Code: Terminal menu → New Terminal) and run:

```
dotnet new console -n DeweyLib
cd DeweyLib
code .
```

Let's unpack exactly what each line did:

`dotnet new console -n DeweyLib` creates a brand new folder called `DeweyLib` containing a working, runnable C# project. `dotnet` is the command-line tool that manages everything about .NET projects — creating them, building them, running them. `new console` tells it which kind of project to create (a console application, as opposed to a website or something else). `-n DeweyLib` sets the project's name.

`cd DeweyLib` moves your terminal “into” that new folder, the same way double-clicking a folder opens it in a file browser.

`code .` opens the current folder in VS Code. The single dot means “this folder right here.”

After running these three commands, look at the file list on the left side of VS Code. You should see:

- `DeweyLib.csproj` — the project file. It records which version of .NET you're using and any extra packages your project depends on. You will rarely need to open this file directly.
- `Program.cs` — this is where your actual code lives. Every C# file ends in `.cs` (short for “C Sharp”). **This is the one file every single step in Milestones 1 through 4 will edit** — so for this whole first part of the guide, you can stop worrying about “which file” and just keep `Program.cs` open.
- `bin/` and `obj/` — folders that VS Code and the compiler create automatically to hold temporary build files. You never edit anything inside them, and it's completely safe to delete them — they'll be recreated the next time you build.

 **If you see:** `dotnet: command not found` → The .NET SDK isn't installed, or your terminal needs to be restarted after installing it. Reinstall from dotnet.microsoft.com and open a fresh terminal.

Step 1.2 — Look at the Starting Code

FILE: Program.cs

This already exists — `dotnet new console` created it for you. Open it; don't create a new one.

Open Program.cs. You'll see it already contains one line:

```
Console.WriteLine("Hello, World!");
```

This is already a complete, runnable program. Let's understand exactly what it does before changing it.

`Console` is the name of a built-in tool in C# that represents the text window your program runs in. Think of it as the object that represents “the screen and keyboard.”

`WriteLine` is an action (formally called a method) that `Console` can perform: print some text, then move to a new line — the same way pressing `Enter` on a keyboard moves you to a new line.

The dot between `Console` and `WriteLine` means “use this specific action that belongs to this specific thing.” You'll see this dot pattern constantly in C#: `thing.action()`.

The text in double quotes, `"Hello, World!"`, is called a string — it's literally a “string of characters.” Anything inside double quotes is treated as literal text, not as code to execute.

The semicolon `;` at the end marks where this instruction (formally called a statement) ends. C# doesn't care about line breaks — it cares about semicolons. Every complete instruction in C# ends with one.

Run the starting program now to see it work:

```
dotnet run
```

You should see `Hello, World!` printed in your terminal. Every time you want to run your program after making changes, you'll type `dotnet run` again.

Step 1.3 — Print a Welcome Banner

FILE: Program.cs

Delete the single `Console.WriteLine("Hello, World!");` line at the top of the file, and put these three lines in its place.

Now let's replace that single line with the real start of `DeweyLib`. Delete the existing line and replace it with:

```
Console.WriteLine(" ");  
Console.WriteLine("  DEWEYLIB v0.1 ");  
Console.WriteLine(" ");
```

Those box-drawing characters might look intimidating, but they're just regular text characters — the same as letters and numbers, just decorative ones used to draw a box around your title. You can also use plain dashes and pipes if you prefer; the box characters are purely cosmetic. The important pattern here is the one you already know: each line is `Console.WriteLine`, some text in quotes, a semicolon.

Run `dotnet run` again. You should see a neat little banner printed at the top of your program's output.

Step 1.4 — Read the User's Name

FILE: Program.cs

Directly below the three banner lines you just added — same file, next lines down.

So far your program only talks — it doesn't listen. Let's fix that. Add these lines after the banner:

```
Console.Write("What's your name? ");  
string name = Console.ReadLine();
```

Notice this uses `Console.Write` (no “Line”) instead of `Console.WriteLine`. The difference is small but important: `Write` prints text and leaves the cursor on the same line, right after what was printed. `WriteLine` prints text and then moves to a new line. We use `Write` here so that whatever the user types appears right after our question, on the same line, the way a normal conversation would look.

`Console.ReadLine()` pauses your program and waits for the user to type something and press Enter. Whatever they typed becomes the result of this instruction.

The word `string` before `name` declares a variable — a labeled storage location that holds a piece of data. `string` tells C# what KIND of data this variable will hold: text. `name` is simply the label we chose; you could call it `userName` or `playerName` just as easily — the name itself doesn't matter to the computer, only to you, as a way to refer back to this piece of data later.

 **Why “string”?** In programming, a sequence of text characters is traditionally called a string — as in “a string of letters,” like beads on a string. You'll see the word `string` constantly; it always means “text.”

Step 1.5 — Greet the User by Name

FILE: Program.cs

Directly below the two lines you just added in Step 1.4.

Now let's use that name. Add this line:

```
Console.WriteLine($"Welcome, {name}!");
```

This single line introduces one of the most useful features in C#: string interpolation. Let's break down exactly what's happening.

Notice the `$` symbol immediately before the opening quote. That dollar sign tells C#: “treat the curly braces inside this text specially — don't print them literally, instead evaluate whatever's inside them and insert the result.”

Inside the string, `{name}` means “insert the current value of the `name` variable right here.” If the user typed Alice, this line behaves exactly as if you had written `Console.WriteLine("Welcome, Alice!");`

Without the dollar sign, C# would print the literal text `Welcome, {name}!` — including the curly braces — because it would have no reason to treat them as anything other than ordinary characters. The dollar sign is what activates the special behavior.

 **Why this matters:** Before string interpolation existed, combining text and data required clunky concatenation: `Console.WriteLine("Welcome, " + name + "!");`. Interpolation

does the same thing, but reads far more naturally — almost like writing a sentence with a blank to fill in.

Step 1.6 — Display the Menu and Read a Choice

FILE: Program.cs

Directly below the welcome line you just added in Step 1.5 — this is the last code you'll add for Milestone 1.

Finally, let's add the menu and capture which option the user picks:

```
Console.WriteLine();  
Console.WriteLine("1. List books");  
Console.WriteLine("2. Check out a book");  
Console.WriteLine("3. Return a book");  
Console.WriteLine("4. Exit");  
Console.Write("Choose an option: ");  
string choice = Console.ReadLine();  
Console.WriteLine($"You chose option {choice}");
```

An empty `Console.WriteLine();` with nothing inside the parentheses simply prints a blank line — useful for visually separating sections of output, the same way you'd leave a blank line between paragraphs.

Everything else here reuses patterns you already learned in this milestone: `WriteLine` to print, `Write` plus `ReadLine` to ask a question and capture the answer, and string interpolation to echo it back.

Step 1.7 — Run It and See the Whole Thing Work

Save the file (Ctrl+S or Cmd+S) and run:

```
dotnet run
```

You should see the banner, a prompt for your name, a greeting using that name, the four-item menu, a prompt for your choice, and finally an echo of whatever you typed. Try running it a few times with different inputs to get comfortable with the flow.

 **If you see:** *nothing happens after I type my name and press Enter* → Make sure you actually pressed Enter after typing — `Console.ReadLine()` waits specifically for the Enter key, not just for you to stop typing.

CHECKPOINT — Milestone 1

- ✓ Running `dotnet run` shows your banner at the top of the output.
- ✓ The program asks for your name and greets you back using that exact name.
- ✓ The four-item menu appears, and whatever number (or text) you type is echoed back.
- ✓ You understand what `Console.WriteLine`, `Console.Write`, `Console.ReadLine`, and `$"..."` each do — you could explain them to someone else in your own words.

 **Reference:** The complete file for this milestone is the starting point inside `Lab_Beginner_DeweyLib.cs`, milestone M1, in the lab files you were given. If your version

doesn't match exactly, that's fine — what matters is that it runs and behaves the way the checkpoint describes.

Milestone 2: Validation & the Main Loop

What You're Building

Right now, your program runs once and stops. A real menu needs to keep showing itself until the user decides to leave, and it needs to handle the inevitable moment when someone types something unexpected — a letter instead of a number, an empty line, or a number outside the valid range. This milestone teaches your program to loop and to defend itself against bad input.

Step 2.1 — Understanding the Loop You Need

FILE: Program.cs

No code to write yet — this step is conceptual. Keep reading before touching the file.

Right now your program's flow is: print menu, read choice, echo choice, then the program ends. We want: print menu, read choice, react to choice, then go right back to printing the menu — forever, until the user picks Exit.

C# has a keyword built exactly for this: `while`. A while loop repeats a block of code for as long as some condition stays true. We're going to use the simplest possible version:

```
while (true)
{
    // everything in here repeats forever
}
```

The word `true` is a fixed value that never changes — it always means “yes.” Since the condition is always true, the loop never stops on its own. That might sound like a mistake, but it's a deliberate, very common pattern: we'll give the user an explicit way to leave (choosing Exit), which uses a different keyword, `return`, to jump out of the loop and end the program entirely.

The curly braces `{` and `}` mark the beginning and end of the block of code that belongs to the loop. Everything between them runs each time the loop repeats. You'll see curly braces used this way constantly in C# — they group statements together.

Step 2.2 — Wrap Your Menu in the Loop

FILE: Program.cs

Take the banner + name/greeting code from Milestone 1, leave it where it is at the top. Wrap **ONLY** the menu-printing and choice-reading lines (Step 1.6's code) in the while loop shown below — this replaces those lines.

Take the menu-printing and choice-reading code from Milestone 1 and move it inside a `while (true)` loop:

```
while (true)
{
    Console.WriteLine();
    Console.WriteLine("1. List books");
    Console.WriteLine("2. Check out a book");
    Console.WriteLine("3. Return a book");
}
```

```
    Console.WriteLine("4. Exit");  
    Console.Write("Choose an option: ");  
    string input = Console.ReadLine();  
}
```

Run this now and you'll see a problem: the menu prints over and over, instantly, without ever waiting for you. That's because we haven't told the program to DO anything different based on what was typed yet — that's the next step.

Step 2.3 — Safely Convert Text Into a Number

FILE: Program.cs

Still inside the while (true) loop you just wrapped in Step 2.2, directly below the string input = Console.ReadLine(); line.

The user's input arrives as a string — plain text — even if they typed a number like 2. Before we can compare it to option numbers, we need to convert that text into an actual number C# can do math and comparisons with. This is where things commonly go wrong for beginners, so let's be careful.

There are two ways to convert text to a number in C#: `int.Parse` and `int.TryParse`. We are only ever going to use the second one, and here's exactly why.

`int.Parse(input)` attempts the conversion and, if the text isn't a valid number (say, the user typed "hello"), it crashes your entire program with an error. That's almost never what you want in a real application — a typo from the user shouldn't kill the whole program.

`int.TryParse(input, out int choice)` attempts the same conversion, but instead of crashing, it returns true or false to tell you whether it worked, and — if it worked — places the resulting number into the choice variable for you to use.

Here's how to use it:

```
if (!int.TryParse(input, out int choice))  
{  
    Console.WriteLine("x Please enter a number.");  
    continue;  
}
```

The exclamation mark ! immediately before `int.TryParse(...)` means "not." So this line reads as: "if the conversion did NOT succeed..." — in other words, this block runs only when the input wasn't a valid number.

The word `out` before `int choice` is a special signal: it tells C# that `TryParse` is going to create this variable and fill it in for us, as a kind of second result alongside the true/false answer. This is the standard pattern used throughout C# for methods that need to return more than one piece of information.

The word `continue` immediately jumps back to the top of the loop, skipping everything else below it in this iteration. We use it here to say: "if the input was invalid, don't bother trying to act on it — just show the menu again."

 **The habit to build now:** Whenever you're about to convert user-typed text into a number, reach for `TryParse`, not `Parse`. This single habit will prevent an enormous number of crashes in your future programs.

Step 2.4 — React to the Choice With a Switch

FILE: Program.cs

Still inside the while loop, directly below the `if (!int.TryParse...)` block you just added in Step 2.3.

Now that we have a valid number safely stored in `choice`, we need to do something different depending on its value. C# offers a tool purpose-built for this: the `switch` statement.

```
switch (choice)
{
    case 1:
        Console.WriteLine("(listing books...)");
        break;
    case 2:
        Console.WriteLine("(checking out...)");
        break;
    case 3:
        Console.WriteLine("(returning...)");
        break;
    case 4:
        return;
    default:
        Console.WriteLine("x Invalid option.");
        break;
}
```

Each case label matches one possible value of `choice`; `break`; stops that case from “falling through” into the next one. `case 4`: calls `return`; — which, as you saw in Step 2.1, exits the loop AND the whole program, since `Exit` should end the application. The `default`: case catches any valid number that isn't 1 through 4 (since `TryParse` only validated that it parsed, not that it's in range — we'll tighten that up in Milestone 3).

CHECKPOINT — Milestone 2

- ✓ Your menu reappears automatically after each choice, instead of the program ending.
- ✓ Typing a letter (like “abc”) shows “X Please enter a number.” and shows the menu again, instead of crashing.
- ✓ Typing 4 ends the program cleanly.
- ✓ You can explain, in your own words, why `int.TryParse` is safer than `int.Parse` for reading user input.

Milestone 3: Refactor Into Methods

What You're Building

Your Main code is probably 30-40 lines long by now, all in one place. That's manageable today, but it will quickly become unreadable as the program grows. This milestone teaches you to extract pieces of that code into methods — named, reusable blocks of code you can call by name instead of retyping.

Think of a method the way you'd think of a recipe in a cookbook: instead of writing out "boil water, add pasta, wait 10 minutes, drain" every single time a recipe needs pasta, you write that once, give it a name ("Cook Pasta"), and just refer to it by name wherever it's needed.

Step 3.1 — Extract the Banner Into a Method

FILE: Program.cs

At the very bottom of the file, below everything else (below your while loop and its closing brace). Methods declared at the top level of Program.cs must go after all the top-level statements, not in the middle of them.

Let's start with the simplest possible method — one that just prints the banner. At the bottom of your file (after all your top-level code), add:

```
static void PrintBanner()  
{  
    Console.WriteLine(" ");  
    Console.WriteLine("  DEWEYLIB v0.3 ");  
    Console.WriteLine(" ");  
}
```

Let's understand every part of this method declaration, piece by piece:

`static` is a keyword we'll explain fully once you reach classes in Milestone 4. For now, just know: every method you write at the top level of Program.cs needs this word in front of it. Think of it as boilerplate you'll always include here.

`void` describes what this method gives back when it's done. `void` specifically means "nothing" — this method performs an action (printing) but doesn't hand back any piece of data when it finishes. You'll see other return types soon that DO hand back a value.

`PrintBanner` is the name we chose for this method. Like variable names, the name itself is just a label for your own benefit — but a clear, descriptive name makes code dramatically easier to read later. The convention in C# is to capitalize the first letter of every word in a method name (called `PascalCase`).

The empty parentheses `()` mean this method doesn't need any information handed to it in order to do its job — it always prints exactly the same banner. You'll see methods with information INSIDE the parentheses very soon.

FILE: Program.cs

Inside your while loop, replace the three `Console.WriteLine(...)` banner calls from Milestone 1 with this one line, in the same spot.

Now replace the three `WriteLine` calls in your main loop with a single line:

```
PrintBanner();
```

Run your program. It should behave identically to before — refactoring (reorganizing code without changing its behavior) should never change what the program actually does, only how the code is organized internally.

Step 3.2 — Extract the Menu-Reading Logic

FILE: Program.cs

At the bottom of the file, alongside PrintBanner() (below it is fine).

Next, let's extract the entire "print menu, read input, validate it" block into its own method — one that loops internally until it gets a valid answer, and only then hands that answer back to whoever called it.

```
static int GetMenuChoice()
{
    while (true)
    {
        Console.WriteLine("\n1. List books");
        Console.WriteLine("2. Check out a book");
        Console.WriteLine("3. Return a book");
        Console.WriteLine("4. Exit");
        Console.Write("Choose an option: ");
        string input = Console.ReadLine();
        if (int.TryParse(input, out int choice) && choice >= 1 && choice
<= 4)
            return choice;
        Console.WriteLine("x Invalid option – enter a number 1-4.");
    }
}
```

This time the return type is int instead of void — this method DOES hand something back: the validated menu choice, as a whole number.

Look closely at `return choice;` inside the while loop. This is a powerful pattern: the moment we have a valid choice, we return it immediately — which also automatically exits the while loop, since `return` ends the whole method right then and there.

The condition `int.TryParse(input, out int choice) && choice >= 1 && choice <= 4` combines three checks with `&&` (which means “and”). All three must be true: the text must convert to a number, AND that number must be 1 or more, AND it must be 4 or less. If any single one of these fails, we fall through to the error message and loop again.

FILE: Program.cs

Replace your entire `while (true) { ... }` main loop body's menu-printing-and-validating section with the single line shown below. Keep the switch statement — just feed it from `GetMenuChoice()` now.

Now your main loop becomes dramatically shorter:

```
while (true)
{
    int choice = GetMenuChoice();
```

```

switch (choice)
{
    // ... your case statements from Milestone 2
}
}

```

 **Notice what happened:** All the messy validation logic moved INSIDE GetMenuChoice. The code that USES the menu choice no longer needs to think about bad input at all — by the time it gets a value back, that value is guaranteed to be valid. This separation — one piece of code handles getting clean data, another piece handles using it — is one of the most valuable habits you can build as a programmer.

Step 3.3 — Extract ListBooks With Parameters

FILE: Program.cs

The two array declarations go just above your while loop (near the top of your top-level code, before the loop starts). The ListBooks method goes at the bottom of the file, with your other methods.

So far our methods haven't needed any outside information. Now let's write one that does. Add this starting data near the very top of your file, before the while loop:

```

string[] titles = { "Dune", "1984", "The Hobbit", "Emma", "It" };
bool[] checkedOut = { false, true, false, false, true };

```

The square brackets after the type — `string[]` and `bool[]` — mean “an array of this type”: a fixed-size, ordered list. `string[] titles` holds five pieces of text. `bool[] checkedOut` holds five true/false values, one for each title, at the matching position. Position 0 in both arrays describes the same book: Dune, currently available (false means “not checked out”).

Now write a method to display this data:

```

static void ListBooks(string[] titles, bool[] checkedOut)
{
    Console.WriteLine();
    for (int i = 0; i < titles.Length; i++)
    {
        string status = checkedOut[i] ? "CHECKED OUT" : "Available";
        Console.WriteLine($"{i + 1}. {titles[i],-20} [{status}]");
    }
}

```

Notice the parentheses now contain `string[] titles, bool[] checkedOut` — these are parameters: named placeholders for information the CALLER must supply when using this method. When you call `ListBooks(titles, checkedOut)` later, whatever arrays you pass in get matched up with these parameter names inside the method.

The for loop is a new kind of loop, useful when you know exactly how many times you want to repeat something — here, once per book. `int i = 0` creates a counting variable starting at zero. `i < titles.Length` is the condition that keeps the loop going (`.Length` tells you how many items are in an array). `i++` increases `i` by exactly 1 after each pass through the loop.

The line `checkedOut[i] ? "CHECKED OUT" : "Available"` uses the ternary operator — a compact way to write a simple if/else as a single expression. Read it as: “if `checkedOut[i]` is true, use the first option; otherwise use the second.”

Inside the interpolated string, `{titles[i], -20}` uses a formatting trick: the `,-20` part means “pad this text out to 20 characters wide, aligned to the left,” which keeps your columns lined up neatly no matter how long each title is.

Step 3.4 — Extract Checkout and Return Logic

FILE: Program.cs

`TryCheckout` and `TryReturn` both go at the bottom of the file with your other methods. The case 2: block they're called from stays inside your switch statement, in the main loop.

Finally, let's extract the checkout and return actions — and introduce one more new idea: a method that needs to hand back TWO pieces of information at once.

```
static bool TryCheckout(string[] titles, bool[] checkedOut, int
bookNumber, out string message)
{
    int index = bookNumber - 1;
    if (index < 0 || index >= titles.Length)
    {
        message = $"There is no book #{bookNumber}.";
        return false;
    }
    if (checkedOut[index])
    {
        message = $"{titles[index]} is already checked out.";
        return false;
    }
    checkedOut[index] = true;
    message = $"{titles[index]} checked out!";
    return true;
}
```

This method returns `bool` (true or false, indicating success or failure) AND uses an `out string message` parameter to hand back a human-readable explanation. This is the exact same `TryParse` pattern you used in Milestone 2 — it's common enough in C# to have a name: the “Try pattern.”

Notice every single path through this method sets `message` before returning. C# actually enforces this — it won't compile if there's any path where an out parameter might be left unset.

Write `TryReturn` as a mirror image — same shape, but checking that the book IS checked out before un-checking it, instead of checking that it ISN'T before checking it out. Try writing this one yourself before checking the lab solution.

FILE: Program.cs

Inside your switch statement, in case 2: — replace the placeholder “(checking out...)” line with the real logic below.

Finally, update your switch statement to call these new methods:

```
case 2:
    Console.Write("Book number to check out: ");
    if (int.TryParse(Console.ReadLine(), out int bookNum))
    {
        if (TryCheckout(titles, checkedOut, bookNum, out string msg))
            Console.WriteLine($"✓ {msg}");
        else
            Console.WriteLine($"✗ {msg}");
    }
    break;
```

✓ CHECKPOINT — Milestone 3

- ✓ Your main loop is now roughly 15 lines long, and reads almost like a table of contents: get a choice, act on it.
- ✓ Trying to check out book #2 (already checked out) shows a clear failure message; checking out book #1 succeeds and the next List shows the change.
- ✓ You can explain what a parameter is and how it's different from a variable declared inside the method.
- ✓ You understand why TryCheckout uses an out parameter instead of just printing the message itself — hint: it keeps the method focused on ONE job (changing data) while leaving the decision of HOW to display the result to the caller.

Milestone 4: Your First Class — Book

What You're Building

The two parallel arrays from Milestone 3 — `titles` and `checkedOut` — work, but they have a serious weakness: nothing forces them to stay in sync. If you ever add a title to one array and forget the other, or insert at different positions, your data quietly becomes corrupted with no warning. This milestone fixes that by introducing a class: a way to bundle related data and the rules that protect it into a single, self-contained unit.

This is genuinely one of the most important ideas in this entire guide. Take your time with it.

Step 4.1 — Why a Class, Specifically?

Imagine instead of two separate arrays, each book was its own self-contained little package, carrying its own title, its own author, and its own checked-out status all together, in one place. That's exactly what a class lets you create.

A class is a blueprint. It describes what every book will have (a title, an author, a checked-out status) and what every book can do (be checked out, be returned). Once you've written that blueprint, you can stamp out as many actual books from it as you want — each one is called an object, or an instance of the class.

Step 4.2 — Write the Book Class

FILE: Program.cs

At the very bottom of the file, after all your existing methods (`PrintBanner`, `GetMenuChoice`, `ListBooks`, `TryCheckout`, `TryReturn`). This is still the same single file — you won't create a separate file for a class until Milestone 5.

Add this at the very bottom of your `Program.cs` file, after all your existing methods:

```
public class Book
{
    public string Title { get; }
    public string Author { get; }
    public bool IsCheckedOut { get; private set; }

    public Book(string title, string author)
    {
        Title = title;
        Author = author;
    }

    public bool Checkout()
    {
        if (IsCheckedOut) return false;
        IsCheckedOut = true;
        return true;
    }
}
```

```

    public bool Return()
    {
        if (!IsCheckedOut) return false;
        IsCheckedOut = false;
        return true;
    }

    public string StatusLine()
        => $"{Title} by {Author} [{(IsCheckedOut ? "OUT" :
"Available")}]";
}

```

This is a lot of new syntax at once. Let's go through every single piece, slowly.

The class declaration

`public class Book` starts the blueprint.

`public` means this class can be used from anywhere in your program (we'll see the alternative, `private`, in a moment). `class` is the keyword that begins a class definition. `Book` is the name we chose — by convention, class names are capitalized, just like method names.

The properties

`public string Title { get; }` declares a property — a named piece of data that belongs to every `Book`. Think of properties as the modern, safer version of the simple variables you used in earlier milestones. The `{ get; }` part says: “anyone can READ this value (get it), but nothing here allows anyone to change it after it's set.” That means once a `Book`'s `Title` is set, it can never be changed again — which makes sense; a book's title shouldn't randomly change while your program is running.

Compare that to `public bool IsCheckedOut { get; private set; }`. This one DOES allow changes — but only from CODE INSIDE THIS CLASS. The word `private` in front of `set` means “only this class's own methods are allowed to assign a new value here.” Code outside the class — your main program, for example — can read `IsCheckedOut` freely, but cannot write `book.IsCheckedOut = true;` directly. It would not even compile if you tried.

🎯 **This is the single most important idea in this entire milestone:** By making `IsCheckedOut` read-only from the outside, we GUARANTEE that the only way it can ever change is by calling `Checkout()` or `Return()` — and those methods are written to enforce the rules correctly, every single time. This is called encapsulation: the object protects its own internal rules, so nothing outside it can corrupt its state by accident.

The constructor

`public Book(string title, string author)` is called a constructor — a special method that runs automatically every time someone creates a new `Book`. Notice it has the same name as the class itself, and no return type at all (not even `void`) — that absence is exactly what marks it as a constructor rather than an ordinary method.

Inside it, `Title = title;` and `Author = author;` copy the values that were passed in into this specific book's own properties. Notice that even though `Title` only allows `get` from outside

the class, the constructor (which IS inside the class) is allowed to set it once, at creation time. After the constructor finishes, Title becomes permanently locked.

The methods

Checkout() and Return() are ordinary methods, just like the ones you wrote in Milestone 3 — except now they live INSIDE the class, and they directly read and write this particular book's own IsCheckedOut property, without needing it passed in as a parameter. Every Book object automatically knows which book it is, and can check and modify its own data directly.

StatusLine() introduces the expression-bodied method syntax: => instead of curly braces, for a method whose entire body is one single expression. `public string StatusLine() => $"...";` means exactly the same thing as writing `public string StatusLine() { return $"..."; }` — it's just a shorter way to write it when the method is simple enough to fit on one line.

Step 4.3 — Replace the Arrays With a List of Books

FILE: Program.cs

Near the top of your file. Delete the two array lines from Milestone 3 (`string[] titles ...` and `bool[] checkedOut ...`) and replace them with the code below, in the same spot.

Now let's actually use this class. First, we need a new tool: `List<Book>`, which is a dynamically-sized collection — unlike the fixed-size arrays from before, you can add and remove items freely.

Replace your two parallel arrays entirely with:

```
var books = new List<Book>
{
    new Book("Dune", "Frank Herbert"),
    new Book("1984", "George Orwell"),
    new Book("The Hobbit", "J.R.R. Tolkien"),
};
```

`var` is a shortcut: instead of writing out the full type (which would be `List<Book>`), `var` tells C# “figure out the type yourself from what's on the right side of the equals sign.” It's purely a convenience for the programmer — the variable still has a real, fixed type underneath, C# just inferred it for you.

Each `new Book("Dune", "Frank Herbert")` calls the constructor you just wrote, creating one actual Book object with those specific values. `new` is the keyword that always means “create a fresh instance of this class.”

FILE: Program.cs

At the very top of the file, above everything else — using statements always go first in a C# file.

You'll need this at the top of your file for List to work:

```
using System.Collections.Generic;
```

A using statement at the top of a file tells C# “I want to use tools that live in this particular toolbox (formally called a namespace), so I don't have to spell out their full location every time.” Without it, you'd have to write the much longer `System.Collections.Generic.List<Book>` every single time you wanted a List.

Step 4.4 — Update ListBooks to Use Book Objects

FILE: Program.cs

Find your existing ListBooks method (with your other methods, near the bottom). Replace its entire signature and body with the version below.

Rewrite ListBooks to accept a List<Book> instead of two arrays, and let each book describe itself:

```
static void ListBooks(List<Book> books)
{
    Console.WriteLine();
    for (int i = 0; i < books.Count; i++)
        Console.WriteLine($"{i + 1}. {books[i].StatusLine()}");
}
```

Notice `books.Count` instead of `titles.Length`. Arrays use `.Length`; Lists use `.Count` — different names for conceptually the same thing (“how many items are in here”), which is a small but common gotcha worth remembering.

`books[i].StatusLine()` reads as: “get the book at position `i`, then ask IT to describe itself.” All the formatting logic that used to live inside ListBooks now lives inside the Book class — ListBooks just asks each book to speak for itself.

Step 4.5 — Update Checkout and Return

FILE: Program.cs

Find your existing TryCheckout method and replace it with the version below. Write TryReturn the same way, right next to it. Then find case 2: and case 3: in your switch statement and update them to pass books (the List<Book>) instead of the two old arrays.

Rewrite TryCheckout to find the right book and call its OWN Checkout method, rather than directly flipping a boolean in an array:

```
static bool TryCheckout(List<Book> books, int bookNumber, out string
message)
{
    if (bookNumber < 1 || bookNumber > books.Count)
    {
        message = $"There is no book #{bookNumber}.";
        return false;
    }
    Book book = books[bookNumber - 1];
    if (book.Checkout())
    {
        message = $"{book.Title}' checked out!";
        return true;
    }
    message = $"{book.Title}' is already checked out.";
    return false;
}
```

 **Notice what disappeared:** There is no longer any code here that directly inspects or modifies whether the book is checked out. That logic now lives entirely inside `Book.Checkout()`, where it belongs. This method's job has narrowed to just one thing: validate the book NUMBER is reasonable, then ask the book itself to handle the rest. If you tried to write `book.IsCheckedOut = true`; here instead, it would not compile — the private set you wrote in Step 4.2 makes that impossible from outside the class.

Write `TryReturn` the same way, calling `book.Return()` instead of `book.Checkout()`. Update your switch statement to call `ListBooks(books)` and pass `books` (the `List<Book>`) everywhere it used to pass the two arrays.

(Optional Challenges)

If you'd like to go further with this milestone, the lab file suggests three extensions. They're optional, but each one teaches something useful:

- Add a `TimesBorrowed` counter that increments every time a book is checked out, and show it in `StatusLine()` — this teaches you to add a second piece of mutable state alongside `IsCheckedOut`, following the exact same private-set pattern.
- Add a menu option to add a new book at runtime, prompting for title and author and calling `books.Add(new Book( . . . ))` — this shows that Lists, unlike arrays, can grow after creation.
- Add a search option using `books.Where(b => b.Title.Contains(term, StringComparison.OrdinalIgnoreCase))` — a preview of LINQ, which gets its own full milestone (M8) very soon.

CHECKPOINT — Milestone 4 — End of the Beginner Tier

- ✓ No parallel arrays remain anywhere in your program — all book data lives inside `Book` objects.
- ✓ Trying to check out an already-checked-out book is blocked BY THE BOOK CLASS ITSELF, not by separate validation code in the menu.
- ✓ You understand why `IsCheckedOut` has a private set, and can explain what would go wrong if it didn't.
- ✓ You can describe, in your own words, the difference between a class (the blueprint) and an object (one specific thing built from that blueprint).

 **Congratulations:** You've completed the Beginner Tier. Save this file — the Intermediate Tier builds directly on top of the `Book` class and `List<Book>` pattern you just created. Take a short break before continuing; Milestone 5 introduces inheritance, which is a genuinely new way of thinking about your code.

PART II

The Intermediate Tier

Milestones 5–8 — Inheritance, generics, file storage, and LINQ

Prerequisite: a completed Milestone 4 (your Book class and List<Book>), or start from the Solutions_Beginner.cs file provided alongside this guide.

Milestone 5: Inheritance & the Media Hierarchy

What You're Building

Real libraries don't just lend books — they lend DVDs, magazines, and other media too. Each type has its own rules (a DVD might be due back in a week; a book in three weeks), but they all share a core idea: something with a title, a barcode, and a checked-out status that can be borrowed and returned.

Without a new tool, you'd be tempted to copy-paste your Book class twice and tweak it for Dvd and Magazine. That would work, but it duplicates code, and any bug you fix in one copy has to be fixed in all three. Inheritance solves this by letting you write the shared behavior ONCE, in a base class, and have Book, Dvd, and Magazine each build on top of it.

 **IN THE PROVIDED SOLUTION** This milestone is the point where the reference solution starts splitting classes into their own files instead of stacking everything in Program.cs. This isn't a change in behavior — it's simply good organization that becomes worthwhile once you have multiple related classes (LibraryItem, Book, Dvd, Magazine). **From here on, every step tells you exactly which file to open or create.**

Step 5.1 — Create the Abstract Base Class

 **NEW FILE: LibraryItem.cs** *(create this file now)*

Create this brand-new file in the same folder as Program.cs (right next to it, not inside a subfolder). Your Book class from Milestone 4 is about to be replaced — delete the old public class Book { ... } block from the bottom of Program.cs once you've moved its ideas into the code below. Program.cs no longer needs its own Book class after this step.

Add this new class to your file (it can replace your existing Book class for now — we'll rebuild Book on top of it in the next step):

```
public abstract class LibraryItem
{
    public string Title { get; }
    public string Barcode { get; }
    public bool IsCheckedOut { get; private set; }
    public DateTime? DueDate { get; private set; }

    protected LibraryItem(string title, string barcode)
    {
        Title = title;
        Barcode = barcode;
    }

    public abstract int LoanDays { get; }
    public virtual decimal DailyLateFee => 0.25m;

    public virtual bool Checkout()
    {
```

```

        if (IsCheckedOut) return false;
        IsCheckedOut = true;
        DueDate = DateTime.Today.AddDays(LoanDays);
        return true;
    }

    public bool Return()
    {
        if (!IsCheckedOut) return false;
        IsCheckedOut = false;
        DueDate = null;
        return true;
    }

    public virtual string StatusLine()
        => $"[{Barcode}] {Title} ({GetType().Name}) " +
            (IsCheckedOut ? $"OUT – due {DueDate:d}" : "Available");
}

```

There's a lot here. Let's take it apart piece by piece.

The word "abstract"

Notice the class is declared as `public abstract class LibraryItem` — not just `public class`. The word `abstract` means this class can never be directly created with `new LibraryItem(...)`. It exists purely as a blueprint FOR OTHER CLASSES to build on — it describes “what every library item has in common,” but the concept of a generic, unspecified “library item” doesn't make sense as an actual real-world object. Only specific things — a `Book`, a `Dvd` — should ever actually be created.

"abstract" members vs. "virtual" members

`public abstract int LoanDays { get; }` declares a property with no implementation at all — just a promise that every subclass (`Book`, `Dvd`, `Magazine`) MUST provide its own value for `LoanDays`. This makes sense: a book's loan period (21 days) is genuinely different from a DVD's (7 days), so there's no sensible “default” to fall back on.

`public virtual decimal DailyLateFee => 0.25m;` is different: it DOES provide a default value (25 cents per day), but the word `virtual` means any subclass is welcome to override it with something different if it needs to (we'll see `Dvd` do exactly that). The distinction is: `abstract` means “you must provide your own”; `virtual` means “here's a default, but feel free to replace it.”

 **How to decide which to use:** Ask yourself: “Is there a sensible default that makes sense for MOST subclasses?” If yes, make it `virtual` with that default. If every subclass genuinely needs something different with no reasonable shared default, make it `abstract`.

The constructor and "protected"

`protected LibraryItem(string title, string barcode)` is the base class's constructor — but notice it says `protected`, not `public`. This is intentional: since `LibraryItem` can never be created directly (it's `abstract`), there's no reason to let outside code call this constructor directly.

protected means “this is only accessible from this class, or from classes that inherit from it” — exactly the access level a base-class constructor should have.

Polymorphism, in one method

Look closely at `Checkout(): DueDate = DateTime.Today.AddDays(LoanDays);`. This line is written ONCE, here, in the base class — but `LoanDays` will resolve differently depending on which actual subclass is calling it. If a `Book` calls `Checkout()`, `LoanDays` means 21. If a `Dvd` calls it, `LoanDays` means 7. The SAME line of code produces different, correct behavior depending on what kind of object is running it.

🎯 **This is polymorphism:** The word literally means “many shapes.” One method, written once, automatically behaves correctly for every different subclass, because each subclass fills in its own version of the parts that need to differ (here, `LoanDays`). You never need to write an if/else checking “is this a book or a DVD?” anywhere — the right behavior just happens.

Step 5.2 — Create the Subclasses

FILE: LibraryItem.cs

Same file, directly below the closing brace of the `LibraryItem` class. All three subclasses live in this one file — they’re small and tightly related to `LibraryItem`, so splitting them into separate files would add overhead without adding clarity.

Now let's build `Book`, `Dvd`, and `Magazine` on top of `LibraryItem`:

```
public class Book : LibraryItem
{
    public string Author { get; }

    public Book(string title, string barcode, string author)
        : base(title, barcode) => Author = author;

    public override int LoanDays => 21;
}

public class Dvd : LibraryItem
{
    public Dvd(string title, string barcode) : base(title, barcode) { }

    public override int LoanDays => 7;
    public override decimal DailyLateFee => 1.00m;
}

public class Magazine : LibraryItem
{
    public int IssueNumber { get; }

    public Magazine(string title, string barcode, int issue)
        : base(title, barcode) => IssueNumber = issue;
```

```

    public override int LoanDays => 3;
}

```

The colon in `public class Book : LibraryItem` means “Book inherits from LibraryItem” — Book automatically gets every public and protected member LibraryItem defined (Title, Barcode, IsCheckedOut, Checkout, Return, and so on), without rewriting any of it.

The constructor line `: base(title, barcode)` calls the BASE class's constructor first, passing along title and barcode, before running anything specific to Book. This is required — every subclass constructor must, directly or indirectly, run the base class's constructor, since the base class is responsible for setting up Title and Barcode.

The keyword `override` in `public override int LoanDays => 21;` is required whenever you're providing your own version of something the base class marked `abstract` or `virtual`. It signals “I'm intentionally replacing the base class's version, not accidentally creating an unrelated new thing with a similar name.” C# will refuse to compile if you write a matching method/property without `override` — this is a deliberate safety check.

Notice Dvd overrides BOTH `LoanDays` (because it's `abstract`, it **MUST**) and `DailyLateFee` (because it **CHOOSES** to, since DVDs cost more per day when late — a dollar instead of a quarter). Book and Magazine don't override `DailyLateFee` at all, so they simply inherit LibraryItem's default of 25 cents.

Step 5.3 — Switch to a Single List<LibraryItem>

FILE: Program.cs

Back in Program.cs (not LibraryItem.cs). Replace your `var books = new List<Book> { ... };` declaration from Milestone 4 with the code below, in the same spot near the top of the file.

Now replace your `List<Book>` with a `List<LibraryItem>`, and seed it with a mix of types:

```

var catalog = new List<LibraryItem>
{
    new Book("Dune", "B-001", "Frank Herbert"),
    new Book("1984", "B-002", "George Orwell"),
    new Dvd("Inception", "D-101"),
    new Dvd("Arrival", "D-102"),
};

```

A `List<LibraryItem>` can hold Book objects, Dvd objects, Magazine objects — any subclass of LibraryItem — all in the SAME list. This works because every Book and every Dvd genuinely IS a LibraryItem (that's exactly what inheritance establishes), so anywhere a LibraryItem is expected, any of its subclasses are accepted too.

Your existing ListBooks-style loop now works completely unchanged, with zero modification, for all the different types at once:

```

foreach (var item in catalog)
    Console.WriteLine(item.StatusLine());

```

 **Try this experiment:** Call `item.Checkout()` on a Book and on a Dvd inside the same loop, then print their `DueDate`. You'll see the Book is due back 21 days from today and the Dvd only

7 days from today — from the exact same line of calling code. That's polymorphism working for you, automatically.

Step 5.4 — Add an Overdue Report

FILE: LibraryItem.cs

Inside the LibraryItem class (not a subclass), just below the DueDate property declaration.

Let's add a simple menu option that finds anything overdue. First, we need a way to know if something IS overdue. Add these two properties inside LibraryItem (just below DueDate):

```
public bool IsOverdue => IsCheckedOut && DueDate < DateTime.Today;  
public int DaysLate => IsOverdue ? (DateTime.Today -  
    DueDate!.Value).Days : 0;
```

Both of these use the expression-bodied => syntax you saw back in Milestone 4's StatusLine — each one is computed FRESH every time you read it, based on the CURRENT date and the item's current state, rather than being calculated once and stored.

The exclamation mark in DueDate!.Value is a different use of ! than the “not” you saw in earlier milestones. Here, it's the null-forgiving operator: it tells the compiler “I know this could theoretically be empty (null), but trust me, in this exact spot I've already confirmed it has a real value, so let me access it directly.” We can make that promise here because IsOverdue already confirmed IsCheckedOut is true, which guarantees DueDate was set.

FILE: Program.cs

Wherever you want to display the report — for example, as a new case in your switch statement, or temporarily right after seeding the catalog to test it.

Now you can scan the catalog for overdue items:

```
foreach (var item in catalog)  
{  
    if (item.IsOverdue)  
        Console.WriteLine($"{item.Title} – {item.DaysLate} days late");  
}
```

FILE: LibraryItem.cs

Inside the LibraryItem class, anywhere among its other methods (Checkout/Return is a natural spot).

To actually test this without waiting weeks, add a small testing helper to LibraryItem:

```
public void BackdateForDemo(int days)  
{  
    if (IsCheckedOut) DueDate = DateTime.Today.AddDays(-days);  
}
```

Check out an item, then call item.BackdateForDemo(10) on it to pretend it's 10 days overdue, and confirm your overdue report finds it.

(Optional Challenge) ReferenceBook

FILE: LibraryItem.cs

Same file, below your Book/Dvd/Magazine classes.

As a stretch goal, the lab suggests a ReferenceBook : Book whose Checkout() always fails, since reference materials never leave the building. To make this possible, Checkout() in LibraryItem needs to be marked virtual (it already is, in the code above), and ReferenceBook can override it:

```
public class ReferenceBook : Book
{
    public ReferenceBook(string title, string barcode, string author)
        : base(title, barcode, author) { }

    public override bool Checkout() => false;
}
```

This is a small but genuinely elegant demonstration of polymorphism: ReferenceBook doesn't need any new if-statements anywhere else in your program to enforce "can't be checked out." It simply overrides the one method responsible for checking out, and every existing call site automatically respects the new rule.

CHECKPOINT — Milestone 5

- ✓ Checking out a Dvd shows a due date 7 days from today; checking out a Book shows 21 days — from the exact same Checkout() call.
- ✓ A single foreach loop over List<LibraryItem> correctly displays Books, DVDs, and (if you add it) Magazines, with zero type-checking code anywhere.
- ✓ You can explain, in your own words, the difference between an abstract member (must override) and a virtual member (may override).
- ✓ You understand why LibraryItem's constructor is protected rather than public.

Milestone 6: Generics & the Repository Interface

What You're Building

Right now, your main program directly manipulates the catalog list — adding to it, searching through it, removing from it. That's fine for a small program, but it tightly couples your menu code to the EXACT way data happens to be stored right now (a `List<LibraryItem>` sitting in memory). What if, later, you wanted to save data to a file, or a real database? You'd have to rewrite every single place that touches catalog directly.

This milestone introduces two related ideas — interfaces and generics — that together let you build ONE reusable storage component that works for ANY type of data, and that can be swapped out later without changing the code that uses it.

Step 6.1 — Understanding Interfaces (Before Any Code)

 **NEW FILE: Repository.cs** *(create this file now)*

No code in this step — read this section, then create the file in Step 6.2.

Think about a wall power outlet. You don't know or care how the electricity behind the wall is generated — you only need ONE thing: a plug that fits the socket. Any device with that matching plug shape works, regardless of what's inside it.

An interface is C#'s way of writing down a "plug shape" for code. It lists WHAT a type must be able to do, without saying anything about HOW. Any class that provides real, working code for everything the interface lists is said to "implement" that interface — the same way any appliance with the right plug shape can use that outlet.

Step 6.2 — Write the Repository Interface

 **FILE: Repository.cs**

This is a new file, separate from `LibraryItem.cs` — a repository is a storage concern, not a media-hierarchy concern, so it earns its own file. Put the interface at the top.

Add this interface to your file:

```
public interface IRepository<T> where T : class
{
    void Add(T item);
    bool Remove(T item);
    T? Find(Func<T, bool> predicate);
    IEnumerable<T> GetAll();
    int Count { get; }
}
```

An interface declaration looks similar to a class, but notice: there is NO code inside any of these members. Just a name and a signature for each one — exactly like the wall outlet only specifying the shape of the plug, not how electricity is made.

Now look at `IRepository<T>`. The angle brackets containing `T` make this a generic interface. `T` is a placeholder — it stands for "whatever type we decide to use this with, decided later." Think of it the way a shipping box works: the same box shape can hold books, or clothes, or

dishes. You design the box once; you decide what goes inside it each time you actually use one.

The clause where `T : class` is a constraint: it restricts `T` to only be a reference type (a class), never a simple value type like `int` or `bool`. We need this constraint because `Find` returns `T?` (a nullable `T`) when nothing matches — and that null-returning behavior only makes sense for reference types.

`Func<T, bool> predicate` is a parameter that holds a SMALL PIECE OF CODE, not a piece of data. Specifically, it represents “any method that takes a `T` and returns true or false.” This lets callers pass in their OWN custom search logic — “find the item whose Barcode matches this string”, for example — without `IRepository` needing to know in advance what kinds of searches it will be asked to do.

Step 6.3 — Implement the Interface

FILE: Repository.cs

Same file, directly below the `IRepository<T>` interface.

Now let's write a class that actually fulfills this promise, backed by a simple in-memory List:

```
public class InMemoryRepository<T> : IRepository<T> where T : class
{
    private readonly List<T> _items = new();

    public int Count => _items.Count;

    public void Add(T item) => _items.Add(item);

    public bool Remove(T item) => _items.Remove(item);

    public T? Find(Func<T, bool> predicate) =>
        _items.FirstOrDefault(predicate);

    public IEnumerable<T> GetAll() => _items.AsReadOnly();
}
```

The colon in `public class InMemoryRepository<T> : IRepository<T>` here means something slightly different than it did with `Book : LibraryItem` in Milestone 5. There, it meant “inherits from.” Here, since `IRepository` is an interface, it means “implements” — `InMemoryRepository` is making a binding promise to provide real, working code for every single member `IRepository` declared. If it forgot one — say, `Count` — the program simply would not compile.

The word `readonly` in front of `private readonly List<T> _items` means this specific variable, once assigned in the constructor (or, as here, at declaration), can never be pointed at a DIFFERENT list later. You can still add and remove items from the list it points to — `readonly` protects the reference itself, not the contents.

The leading underscore in `_items` is a common, widely-used naming convention for private fields in C# — it visually distinguishes “internal storage this class manages” from ordinary local variables and parameters, purely as a readability aid.

`FirstOrDefault(predicate)` is a method that searches through `_items` and returns the first one where calling `predicate` on it returns `true` — or, if nothing matches, returns the “default” value for that type, which for a reference type is `null`. This is part of LINQ, which you’ll explore fully in Milestone 8.

 FILE: Repository.cs

Same file, at the very top — using statements always go first.

using `System`; and using `System.Collections.Generic`; and using `System.Linq`; at the top, for `Func`, `List`, and `FirstOrDefault` respectively.

Step 6.4 — Use the Repository in Your Program

 FILE: Program.cs

Near the top of the file. Replace your `var catalog = new List<LibraryItem> { ... }` line from Milestone 5 with the code below — same spot, same purpose, different storage.

Replace your `catalog` list declaration with:

```
IRepository<LibraryItem> catalog = new InMemoryRepository<LibraryItem>();
catalog.Add(new Book("Dune", "B-001", "Frank Herbert"));
catalog.Add(new Dvd("Inception", "D-101"));
```

 **Read this line very carefully:** `IRepository<LibraryItem> catalog = new InMemoryRepository<LibraryItem>()`; The variable’s declared type, on the left, is the **INTERFACE** — not the concrete class. The actual object being created, on the right, is the concrete class. This is one of the most important habits to build in all of object-oriented programming: declare variables using the most general type that still does what you need, and reserve the specific concrete type for the single moment you create the object.

Ask yourself: what would have to change if, six months from now, you wanted to swap in a database-backed repository instead? The answer: exactly one line — the new `InMemoryRepository<LibraryItem>()` on the right. Every other place in your program that uses `catalog` — searching it, listing it, adding to it — continues to work completely unchanged, because all of that code only ever talked to the `IRepository<LibraryItem>` promise, never to `InMemoryRepository` specifically.

 FILE: Program.cs

Throughout the rest of `Program.cs` — anywhere you previously called `catalog.Add(...)`, looped over it directly, or searched it by hand.

Update the rest of your menu code to call `catalog.Add(...)`, `catalog.Find(...)`, and `catalog.GetAll()` instead of working with a raw list directly.

Step 6.5 — Add a Second Repository for Members

 FILE: Repository.cs

Same file, below `InMemoryRepository<T>`. The `Member` record is small enough that it doesn’t need its own file.

This is where the payoff of generics becomes vivid. Add a simple record to represent a library member:

```
public record Member(int Id, string Name, string Email);
```

A record is a compact way to declare a simple class meant mainly to hold data. `public record Member(int Id, string Name, string Email);` automatically generates a constructor, three get-only properties, and useful comparison behavior — all from this single line. It's a convenient shorthand for the kind of small, data-focused class you wrote by hand in Milestone 4.

FILE: Program.cs

Anywhere after your catalog declaration — these few lines can sit right beneath it.

Now add a second repository, for an entirely different type of data:

```
IRepository<Member> members = new InMemoryRepository<Member>();  
members.Add(new Member(1, "Alice", "alice@example.com"));  
members.Add(new Member(2, "Bob", "bob@example.com"));
```



Notice what just happened: You did not write a single new line of storage logic. The exact same `InMemoryRepository<T>` class — written once, for any type `T` — now manages an entirely different kind of data, with full type safety. `members.Add("not a member")` would not even compile, because `T` was fixed to `Member` the moment you wrote `InMemoryRepository<Member>()`. This is the entire point of generics: write a behavior once, reuse it safely for any type.



CHECKPOINT — Milestone 6

- ✓ Your program's behavior is identical to Milestone 5, but all catalog access now flows through `IRepository<LibraryItem>` rather than a raw list.
- ✓ Adding a second repository for `Member` took only a few lines, because `InMemoryRepository<T>` is generic.
- ✓ You can explain why catalog is declared as `IRepository<LibraryItem>` rather than `InMemoryRepository<LibraryItem>`, and what flexibility that buys you.
- ✓ You understand the difference between a class implementing an interface (a binding promise, checked by the compiler) and a class inheriting from a base class.

Milestone 7: Exceptions & File Persistence

What You're Building

Two separate problems need solving in this milestone. First: right now, when something goes wrong — like checking out a barcode that doesn't exist — your code returns true/false and an out message, the same pattern from Milestone 3. That works, but as your program grows, richer error information becomes valuable. Second: every time you close your program, all your data vanishes — there is no actual library if nothing survives a restart. This milestone fixes both: we introduce exceptions for rich error handling, and JSON file storage so your catalog survives between runs.

Part A: Custom Exceptions

Step 7.1 — Understanding Exceptions (Before Any Code)

Think of a fire alarm. When something goes wrong in a building, you don't quietly note it and continue — you raise a loud, specific, structured alarm: "Fire on the 3rd floor, east wing." An exception in C# is exactly that: a structured, detailed object that says "something went wrong, and here is precisely what and where."

You've already been using built-in exceptions without necessarily naming them — if you ever tried `int.Parse` on bad text, the crash you saw was an exception being thrown without anything catching it. In this milestone we'll create our OWN exception types, carrying exactly the information we need, and we'll catch them deliberately instead of letting them crash the program.

Step 7.2 — Create Two Custom Exceptions

 **NEW FILE: Exceptions.cs** *(create this file now)*

Create this brand-new file, next to `LibraryItem.cs` and `Repository.cs`. Both exception classes go in this one file.

```
public class ItemNotFoundException : Exception
{
    public string Barcode { get; }

    public ItemNotFoundException(string barcode)
        : base($"No item with barcode '{barcode}'.") => Barcode = barcode;
}

public class ItemUnavailableException : Exception
{
    public string Title { get; }
    public DateTime? DueBack { get; }

    public ItemUnavailableException(string title, DateTime? dueBack,
        string reason)
        : base($"'{title}' {reason}")
```

```

    {
        Title = title;
        DueBack = dueBack;
    }
}

```

The colon : Exception here means `ItemNotFoundException` INHERITS from the built-in Exception class — the same inheritance mechanism from Milestone 5, just applied to one of C#'s own built-in types instead of one you wrote yourself. This automatically gives your custom exception everything a normal exception needs (a `Message` property, stack trace tracking, and so on).

Just like `LibraryItem`'s constructor in Milestone 5, this constructor calls : `base(...)` — here, passing a friendly, formatted message UP to the built-in Exception class's own constructor, which is what fills in the standard `.Message` property every exception has.

The key idea that makes this exception more useful than a generic one: it carries its OWN extra data — `Barcode` on one, `Title` and `DueBack` on the other. Code that catches these exceptions later can read these properties directly, rather than having to parse information back out of a plain text message.

Step 7.3 — Build a CheckoutService That Throws

 **NEW FILE: CheckoutService.cs** *(create this file now)*

Create this brand-new file, next to `Exceptions.cs`.

Now let's create a dedicated class whose entire job is handling the business rules around checking items in and out — and which raises our new exceptions when something's wrong, instead of returning true/false.

```

public class CheckoutService
{
    private readonly IRepository<LibraryItem> _catalog;

    public CheckoutService(IRepository<LibraryItem> catalog) => _catalog =
catalog;

    public LibraryItem Borrow(string barcode)
    {
        var item = _catalog.Find(i => i.Barcode == barcode)
            ?? throw new ItemNotFoundException(barcode);

        if (!item.Checkout())
            throw new ItemUnavailableException(item.Title, item.DueDate,
"is already checked out");

        return item;
    }

    public LibraryItem GiveBack(string barcode)

```

```

    {
        var item = _catalog.Find(i => i.Barcode == barcode)
            ?? throw new ItemNotFoundException(barcode);

        if (!item.Return())
            throw new ItemUnavailableException(item.Title, null, "was not
checked out");

        return item;
    }
}

```

The pattern `_catalog.Find(...) ?? throw new ItemNotFoundException(barcode);` combines two things you've already learned. `??` (from Milestone 1's discussion of nullable types) provides a fallback when the left side is null. Here, instead of a normal fallback VALUE, we provide a throw expression — meaning “if Find comes back empty, immediately raise this exception instead of continuing.” This is a compact, very common C# idiom: “find this, or blow up with a specific, descriptive error.”

Notice this class doesn't print anything to the console, and doesn't know or care HOW errors will eventually be displayed to a user. It only enforces RULES, and communicates failure by throwing. This separation matters: the exact same CheckoutService will work completely unchanged later when we build a Web API in Milestone 12, even though a web API needs to turn errors into HTTP responses rather than console text.

Step 7.4 — Catch Exceptions in Your Menu

FILE: Program.cs

Near the top, alongside your other setup lines (catalog, members), add the new `var checkoutService = ...` line. Then find case 2: (checkout) in your switch statement and replace its body with the try/catch version below. Write the mirror-image version for case 3: (return).

Now update your menu code to use CheckoutService, catching the specific exceptions it might throw:

```

var checkoutService = new CheckoutService(catalog);

// ... inside your menu's checkout case:
Console.Write("Barcode: ");
try
{
    var item = checkoutService.Borrow(Console.ReadLine() ?? "");
    Console.WriteLine($"✓ '{item.Title}' due back {item.DueDate:d}");
}
catch (ItemNotFoundException ex)
{
    Console.WriteLine($"✗ {ex.Message} (you typed: {ex.Barcode})");
}
catch (ItemUnavailableException ex)
{

```

```
Console.WriteLine($"x {ex.Message}");  
}
```

A try block wraps code that MIGHT throw an exception. Each catch (SomeExceptionType ex) block following it handles ONE specific kind of exception, if it occurs. C# checks each catch block in order and runs the first one whose type matches what was actually thrown.

Notice we can use ex.Barcode in the first catch block — that's the custom property we added to ItemNotFoundException in Step 7.2. This is exactly the payoff of writing custom exceptions: we get to use rich, specific information when handling the error, not just a generic text message.

 **The design rule to remember:** Services throw; user interfaces catch. CheckoutService's job is to enforce rules and describe failures precisely. The menu code's job is to decide HOW to present those failures to a human. Keeping these two responsibilities separate means the exact same CheckoutService can work underneath a console menu, a web API, or any other interface you build in the future — without changing a single line inside CheckoutService itself.

Part B: JSON File Persistence

Step 7.5 — Why JSON?

JSON (JavaScript Object Notation, despite the name, used everywhere, not just in JavaScript) is a simple, human-readable text format for representing structured data. We're going to use it to save your entire catalog to a file on disk, so it survives between runs of your program.

There's a wrinkle: your catalog contains a MIX of types (Book, Dvd, Magazine), all stored as the common LibraryItem base type. JSON doesn't automatically know which specific subclass each saved item should become when loaded back. We solve this with a "Kind" field that records which subclass each item actually was.

Step 7.6 — Create a Flat Data Shape for Saving

 **NEW FILE: LibraryStorage.cs** *(create this file now)*

Create this brand-new file, next to CheckoutService.cs. The ItemData record and the LibraryStorage class below both go in this one file.

```
public record ItemData(  
    string Kind, string Title, string Barcode,  
    bool IsCheckedOut, DateTime? DueDate,  
    string? Author = null, int? IssueNumber = null);
```

This record is a deliberately FLAT shape — it doesn't use inheritance at all, even though our actual LibraryItem hierarchy does. It exists purely as a simple, neutral format for writing to and reading from a file. Kind will hold the literal text "Book", "Dvd", or "Magazine", telling us which real class to reconstruct when loading.

The = null after Author and IssueNumber makes these parameters optional with a default value — the same optional-parameter idea from Milestone 3's CalculateFee example. A Dvd being saved doesn't have an Author or IssueNumber, so those fields are simply left at null for that record.

Step 7.7 — Write the Save and Load Methods

FILE: LibraryStorage.cs

Same file, directly below the ItemData record.

```
public static class LibraryStorage
{
    public static void Save(IEnumerable<LibraryItem> items, string path)
    {
        var dtos = items.Select(i => i switch
        {
            Dvd d => new ItemData("Dvd", d.Title, d.Barcode,
d.IsCheckedOut, d.DueDate),
            Magazine m => new ItemData("Magazine", m.Title, m.Barcode,
m.IsCheckedOut, m.DueDate, IssueNumber: m.IssueNumber),
            Book b => new ItemData("Book", b.Title, b.Barcode,
b.IsCheckedOut, b.DueDate, b.Author),
            _ => throw new NotSupportedException()
        });

        var options = new JsonSerializerOptions { WriteIndented = true };
        File.WriteAllText(path, JsonSerializer.Serialize(dtos, options));
    }

    public static List<LibraryItem> Load(string path)
    {
        if (!File.Exists(path)) return new();

        var json = File.ReadAllText(path);
        var dtos = JsonSerializer.Deserialize<List<ItemData>>(json) ??
new();

        var items = new List<LibraryItem>();
        foreach (var d in dtos)
        {
            LibraryItem item = d.Kind switch
            {
                "Book" => new Book(d.Title, d.Barcode, d.Author ??
"Unknown"),
                "Dvd" => new Dvd(d.Title, d.Barcode),
                "Magazine" => new Magazine(d.Title, d.Barcode,
d.IssueNumber ?? 0),
                _ => throw new NotSupportedException()
            };
            items.Add(item);
        }
        return items;
    }
}
```

```
}  
}
```

FILE: LibraryStorage.cs

Same file, at the very top, above the ItemData record.

using System.IO; and using System.Text.Json; (and using System.Linq; for Select) to the top of this file.

The expression `i switch { Dvd d => ..., Magazine m => ..., Book b => ..., _ => ... }` is a switch EXPRESSION (different from the switch STATEMENT you used in Milestone 2). Instead of running different statements for different cases, it evaluates to different VALUES depending on the actual runtime type of `i`. This is C#'s pattern matching feature — it can check “what concrete type is this object actually?”, not just “what value does this variable hold?”

`JsonSerializer.Serialize(dtos, options)` converts our list of plain ItemData records into JSON text. `JsonSerializer.Deserialize<List<ItemData>>(json)` does the reverse: takes JSON text and reconstructs a list of ItemData objects from it. Both live in the built-in System.Text.Json namespace, which is why we added that using statement at the top.

Notice `static class LibraryStorage` — the word `static` on a CLASS (not just a method) means this class can never be instantiated with `new` at all; it exists purely as a named container for related utility methods, called directly as `LibraryStorage.Save(...)` without ever creating a `LibraryStorage` object.



IN THE PROVIDED SOLUTION The Load method shown above (straight from the lab text) rebuilds each item but doesn't restore its checked-out state or due date — every item comes back marked Available. That actually breaks the very checkpoint at the end of this milestone (“still checked out after a restart”). The reference solution's Load adds a short block after creating each item that calls `item.Checkout()` and then restores the exact saved DueDate using `BackdateForDemo` (from Milestone 5) when `d.IsCheckedOut` is true. If your own Load doesn't do this, your checkpoint test below will appear to fail even though everything else is correct — add that restoration block before moving on.

Step 7.8 — Load at Startup, Save After Every Change

FILE: Program.cs

Near the top of the file, right after creating your catalog and checkoutService. This replaces however you were seeding the catalog with sample data in earlier milestones — the load-or-seed logic goes in this one spot.

```
string dataPath = Path.Combine(AppContext.BaseDirectory, "deweylib.json");  
try  
{  
    var loaded = LibraryStorage.Load(dataPath);  
    if (loaded.Count > 0)  
        foreach (var item in loaded) catalog.Add(item);  
    else  
        SeedCatalog(catalog); // your existing sample-data method  
}  
catch (Exception ex)
```

```
{  
    Console.WriteLine($"⚠ Could not load saved data ({ex.Message}).  
Starting fresh.");  
    SeedCatalog(catalog);  
}
```

Wrapping the load attempt in try/catch is important: if the JSON file ever becomes corrupted (say, someone edits it by hand and breaks the formatting), Deserialize will throw an exception. Without this try/catch, that would crash your ENTIRE program on startup, every time, with no way to recover. Catching it here lets us fall back gracefully to fresh sample data instead.

FILE: Program.cs

After every successful checkout and return inside your try blocks from Step 7.4, and once more right before the program exits (in your Exit case).

Finally, call `LibraryStorage.Save(catalog.GetAll(), dataPath)`; after every successful checkout, every successful return, and when the program exits, so changes are never lost.

CHECKPOINT — Milestone 7

- ✓ Checking out an item, closing the program, and running it again shows that item is still checked out.
- ✓ Typing a barcode that doesn't exist shows a friendly message that includes the exact barcode you typed.
- ✓ Manually deleting or corrupting the JSON file and restarting the program shows a warning and starts fresh, without crashing.
- ✓ You can explain why CheckoutService throws exceptions instead of returning true/false, and why that decision was made HERE specifically, even though Milestone 3's TryCheckout used the boolean approach successfully.

Milestone 8: LINQ Reports Dashboard

What You're Building

Your catalog is now a real, persistent collection of data. This milestone introduces LINQ (Language Integrated Query) — a built-in C# toolkit for searching, filtering, sorting, and summarizing collections, using compact, readable code instead of hand-written loops with if-statements.

Imagine sorting through a box of mixed photographs by hand: checking each one's date, setting aside the ones from a certain year, then sorting that smaller pile. That's exactly what a loop with conditions does in code. LINQ does the same job, but in a single, readable line, that often reads almost like an English sentence describing what you want.

Step 8.1 — Your First LINQ Query: Where and OrderBy

 **NEW FILE: Reports.cs** *(create this file now)*

Create this brand-new file, next to LibraryStorage.cs. Everything in this milestone (all six reports) goes in this one new file — not in Program.cs. Program.cs will just call into it.

Add this using statement near the top of your file if it isn't already there:

```
using System.Linq;
```

Now write Report 1: available items, alphabetical by title.

```
var available = catalog.GetAll()
    .Where(i => !i.IsCheckedOut)
    .OrderBy(i => i.Title);

foreach (var item in available)
    Console.WriteLine(item.Title);
```

`.Where(i => !i.IsCheckedOut)` filters the collection, keeping only items where the condition inside the parentheses is true. `i => !i.IsCheckedOut` is a lambda expression — a compact way of writing a small, unnamed function. Read it as: “for each item *i*, check whether NOT `IsCheckedOut`.” It's conceptually identical to the predicate parameter you wrote for `IRepository.Find` back in Milestone 6 — in fact, LINQ's `Where` uses the exact same idea.

`.OrderBy(i => i.Title)` sorts the results, in this case alphabetically by `Title`, since strings sort alphabetically by default.

Notice these two operations are CHAINED with dots, one after another:

`catalog.GetAll().Where(...).OrderBy(...)`. Each operation takes a collection and produces a new, transformed collection, which the next operation in the chain immediately works on. You can chain as many LINQ operations together as you need.

Step 8.2 — Projection With Select

 **FILE: Reports.cs**

Same file, as a new report alongside Report 1.

Report 2 needs to show items checked out, sorted by due date, with the number of days remaining calculated for each one:

```
var dueSoonest = catalog.GetAll()
    .Where(i => i.IsCheckedOut)
    .OrderBy(i => i.DueDate)
    .Select(i => new { i.Title, Days = (i.DueDate!.Value -
DateTime.Today).Days });

foreach (var x in dueSoonest)
    Console.WriteLine($"{x.Title}: {x.Days} days left");
```

.Select(...) transforms each item into something new — in LINQ terminology, this is called "projection." Here, instead of working with the full `LibraryItem` object, we project each one into a small, temporary shape containing just a `Title` and a calculated `Days` value.

The expression `new { i.Title, Days = ... }` creates an anonymous type — a lightweight, unnamed object created purely to carry exactly these two pieces of data for the rest of this query. You'll never see "anonymous type" written anywhere as an actual class declaration; C# generates it for you, on the spot, based on this exact shape.

Step 8.3 — Grouping: The Most Important LINQ Operator

FILE: Reports.cs

Same file, as a new report.

Report 3 needs to break the catalog down by media type. This is where `GroupBy` comes in — take a moment with this one, since it's genuinely the trickiest LINQ concept to visualize at first.

Imagine taking your entire catalog and sorting it into separate piles — one pile of Books, one pile of DVDs, one pile of Magazines. `GroupBy` does exactly that: it doesn't just sort, it BUNDLES matching items together into named groups.

```
var byType = catalog.GetAll().GroupBy(i => i.GetType().Name);

foreach (var group in byType)
    Console.WriteLine($"{group.Key}: {group.Count()} items");
```

`GroupBy(i => i.GetType().Name)` sorts every item into a pile based on the result of `i.GetType().Name` — which returns the actual runtime class name ("Book", "Dvd", or "Magazine") for each item. Items that produce the same result end up in the same pile.

Each group you get back from a `foreach` over the `GroupBy` result has TWO things: a `.Key` (the value everything in this pile shares — here, the type name) and the actual collection of items in that pile, which you can enumerate or call `.Count()` on, exactly like any other collection.

You can go further and print the actual titles inside each group:

```
foreach (var group in byType)
{
    Console.WriteLine($"{group.Key}:");
    foreach (var item in group)
        Console.WriteLine($"  - {item.Title}");
}
```

Step 8.4 — Aggregates: Sum, Count, MaxBy

FILE: Reports.cs

Same file, as a new report.

Report 4 needs a fee summary across all overdue items:

```
var overdue = catalog.GetAll().Where(i => i.IsOverdue).ToList();
Console.WriteLine($"Overdue items: {overdue.Count}");
Console.WriteLine($"Total fees: {overdue.Sum(i => i.CurrentFee):C}");

var worst = overdue.MaxBy(i => i.DaysLate);
Console.WriteLine($"Worst: {worst?.Title} ({worst?.DaysLate} days)");
```

 **IN THE PROVIDED SOLUTION** This report calls `i.CurrentFee`, a property the lab text uses here but never actually shows you how to write. Add it to `LibraryItem.cs` yourself, right next to `DaysLate` (Milestone 5):

```
public decimal CurrentFee => DaysLate * DailyLateFee;
```

This multiplies how many days late an item is by its per-day fee (`DailyLateFee`, which you wrote back in Milestone 5 — 25¢ for Books and Magazines, \$1.00 for Dvds). Without this property, Step 8.4's code above won't compile.

`.Sum(i => i.CurrentFee)` adds up the result of `CurrentFee` across every item in the collection, producing a single total number.

`.MaxBy(i => i.DaysLate)` finds the SINGLE item with the highest `DaysLate` value — the worst offender, in this case. This is different from `Max`, which would give you back just the highest NUMBER, not the item that produced it.

Notice `worst?.Title` — the question mark before the dot is the null-conditional operator. If `overdue` happened to be empty, `MaxBy` would return null, and `worst?.Title` safely evaluates to null too, instead of crashing trying to read `.Title` off of nothing.

 **The #1 LINQ gotcha: deferred execution** When you write `var query = catalog.GetAll().Where(...)`, the filtering does NOT happen on that line. LINQ queries are lazy — the actual work only happens when you enumerate the result (with `foreach`, or by calling `.ToList()`, `.Count()`, etc). This means if the underlying data changes between defining the query and actually using it, your query will see the CHANGED data. When you need a frozen snapshot at a specific moment, call `.ToList()` immediately to force the query to run right then.

Step 8.5 — Conditional Queries

FILE: Reports.cs

Same file, as the final (sixth) report.

Report 6 (search) needs to apply filters only if the user actually provided them. This is a useful, very common LINQ pattern — building up a query conditionally:

```
var query = catalog.GetAll();
```

```

if (!string.IsNullOrEmpty(titleFragment))
    query = query.Where(i => i.Title.Contains(titleFragment,
StringComparison.OrdinalIgnoreCase));

if (!string.IsNullOrEmpty(typeFilter))
    query = query.Where(i => i.GetType().Name.Equals(typeFilter,
StringComparison.OrdinalIgnoreCase));

foreach (var item in query)
    Console.WriteLine(item.StatusLine());

```

Thanks to deferred execution (explained above), this works perfectly: each `query = query.Where(...)` line doesn't run the filter immediately — it just adds ANOTHER step onto the existing un-run query. By the time the `foreach` finally enumerates it, every filter that was conditionally added gets applied, all at once.

Putting It Together: A Reports Menu

FILE: Program.cs

In your `switch` statement — add a new case (for example, case 4: "Reports") that calls into the report logic you wrote in `Reports.cs`. If you wrote your six reports as separate methods inside `Reports.cs`, call them from a small submenu here.

Wrap your six reports in a menu, matching the structure you've used since Milestone 2 — print options, read a choice, switch on it. By now this pattern should feel completely familiar.

CHECKPOINT — Milestone 8 — End of the Intermediate Tier

- ✓ All six reports run against your live, persisted catalog data and reflect changes immediately after a checkout or return.
- ✓ You used `GroupBy` at least twice and can explain, using the "piles" analogy or your own words, what it actually does.
- ✓ You can demonstrate deferred execution to someone else — show them a query producing different results depending on when it's enumerated.
- ✓ You understand why `Sum`, `Count`, and `MaxBy` are called "aggregates" — they each collapse an entire collection down into one single result.

 **Congratulations:** You've completed the Intermediate Tier. Your console application now has a real object hierarchy, generic storage, proper error handling, file persistence, and powerful querying. The Advanced Tier ahead adds events, asynchronous programming, reflection, and finally a Web API — turning this console app into something other programs can talk to over the internet.

PART III

The Advanced Tier

Milestones 9–12 — Events, async programming, reflection, and a REST API

Prerequisite: a completed Milestone 8, or start from `Solutions_Intermediate.cs`.

Milestone 9: Events & Notifications

What You're Building

Think about a doorbell. When someone presses it, the doorbell doesn't know or care who reacts — maybe a dog barks, maybe someone walks to the door, maybe a security camera starts recording. The person who installed the doorbell didn't need to know in advance everything that would eventually respond to it. You could add a brand new smart camera years later, and the doorbell still works exactly the same way.

Right now, your CheckoutService just performs its job silently. This milestone teaches it to ANNOUNCE when something happens — a checkout, a return — so that other, completely independent parts of your program can react, without CheckoutService ever needing to know they exist.

Step 9.1 — Building Up to Events: Storing a Method in a Variable

FILE: Program.cs

This is a throwaway snippet to try out the IDEA, not something you need to keep. Type it temporarily near the top of your file to see it run, then delete it before moving on — Step 9.2 is the real code for this milestone.

Before writing an event, it helps to see the simpler building block underneath it. In C#, you can store a reference to a METHOD in a variable, the same way you'd store a number or a string. This is called a delegate.

```
Action<string> notify = message => Console.WriteLine(message);  
notify("Hello!"); // prints "Hello!"
```

Action<string> is a built-in delegate type representing “any method that takes one string parameter and returns nothing.” message => Console.WriteLine(message) is a lambda expression (you've seen this syntax since Milestone 8's LINQ queries) defining exactly such a method, right on the spot, without giving it a separate name.

Crucially, you can ADD a second method to the same variable using +=:

```
notify += message => Console.WriteLine($"LOGGED: {message}");  
notify("Book returned"); // BOTH lines run now
```

This is called a multicast delegate: one call to notify(...) now runs every method that's been added with +=, in the order they were added. This is the exact mechanism events are built on top of.

Step 9.2 — Define the Event

NEW FILE: Events.cs *(create this file now)*

Create this brand-new file, next to LibraryStorage.cs. CirculationEventArgs goes here.

An event is a safer, more controlled version of the multicast delegate idea — specifically designed so that outside code can only ADD or REMOVE handlers (using += and -=), never directly trigger the event itself. Only the class that OWNS the event is allowed to do that.

First, define a class to carry information about what happened:

```

public class CirculationEventArgs : EventArgs
{
    public LibraryItem Item { get; }
    public string Action { get; }
    public DateTime Timestamp { get; }

    public CirculationEventArgs(LibraryItem item, string action)
    {
        Item = item;
        Action = action;
        Timestamp = DateTime.Now;
    }
}

```

Inheriting from the built-in EventArgs class is a long-standing C# convention for any class meant to carry information ABOUT an event that just occurred. It doesn't add any required functionality — it's mainly a signal of intent, recognized throughout the .NET ecosystem.

FILE: CheckoutService.cs

Open your existing CheckoutService.cs (from Milestone 7). Add the event line shown below inside the class, and add the two ItemCirculated?.Invoke(...) lines inside Borrow and GiveBack respectively, right before each method's final return item; line. Everything else in this file from Milestone 7 stays exactly as it was.

Now add the event itself to CheckoutService:

```

public class CheckoutService
{
    // ... existing fields and constructor from Milestone 7 ...

    public event EventHandler<CirculationEventArgs>? ItemCirculated;

    public LibraryItem Borrow(string barcode)
    {
        // ... existing logic from Milestone 7 ...
        ItemCirculated?.Invoke(this, new CirculationEventArgs(item,
"checkout"));
        return item;
    }

    public LibraryItem GiveBack(string barcode)
    {
        // ... existing logic ...
        ItemCirculated?.Invoke(this, new CirculationEventArgs(item,
"return"));
        return item;
    }
}

```

The keyword `event` in front of `EventHandler<CirculationEventArgs>? ItemCirculated;` is what makes this special, compared to the plain `Action` variable from Step 9.1. With event, code OUTSIDE this class can only ever write `service.ItemCirculated += someHandler;` or `-=`. It is not even legal to write `service.ItemCirculated.Invoke(...)` from outside — that's a compile error. Only `CheckoutService` itself, from inside its own methods, is allowed to call `Invoke`.

The question mark immediately after `Invoke` (`ItemCirculated?.Invoke(...)`) is the null-conditional operator again, from Milestone 5. If nobody has subscribed to this event yet, `ItemCirculated` is null, and `?.Invoke(...)` safely does nothing instead of crashing. This check is required — raising an event with zero subscribers is a completely normal, valid situation.

 **Why this matters:** `CheckoutService.Borrow()` and `GiveBack()` haven't changed their core logic at all from Milestone 7 — they've simply gained one new line each, announcing what happened. `CheckoutService` still knows absolutely nothing about WHO is listening, or HOW MANY listeners there are, or what they'll do in response. That knowledge lives entirely outside this class, which is exactly what keeps it reusable.

Step 9.3 — Wire Up Independent Subscribers

FILE: Program.cs

Right after the line where you create `checkoutService` (`var checkoutService = new CheckoutService(catalog);`), still near the top of the file.

Now, in your program's startup code (outside `CheckoutService` entirely), subscribe several completely independent reactions:

```
var checkoutService = new CheckoutService(catalog);

// Subscriber 1: print to console
checkoutService.ItemCirculated += (sender, e) =>
    Console.WriteLine($"🔔 {e.Item.Title} {e.Action} at {e.Timestamp:t}");

// Subscriber 2: append a line to a log file
checkoutService.ItemCirculated += (sender, e) =>
    File.AppendAllText("library.log", $"{e.Timestamp:u} {e.Action} {e.Item.Barcode}\n");
```

Both subscribers use a lambda expression matching the shape `EventHandler<CirculationEventArgs>` expects: two parameters, conventionally named `sender` (the object that raised the event — here, the `CheckoutService` itself) and `e` (the event-specific data — here, our `CirculationEventArgs`).

Now call `checkoutService.Borrow("B-001")` somewhere and watch BOTH subscribers fire automatically, from that one call — one printing to the console, one writing to a file, with `CheckoutService` itself completely unaware either one exists.

Step 9.4 — The Hold Watcher: A Subscriber That Has Its Own Event

FILE: Events.cs

Same file as `CirculationEventArgs`. Add `HoldReadyEventArgs` and `HoldWatcher` below it.

Let's build something more interesting: a HoldWatcher that lets patrons "reserve" an unavailable item, and gets notified automatically the moment it's returned.

```
public class HoldReadyEventArgs : EventArgs
{
    public string Patron { get; }
    public LibraryItem Item { get; }

    public HoldReadyEventArgs(string patron, LibraryItem item)
    { Patron = patron; Item = item; }
}

public class HoldWatcher
{
    private readonly Dictionary<string, Queue<string>> _holds = new();

    public event EventHandler<HoldReadyEventArgs>? HoldReady;

    public void PlaceHold(string barcode, string patron)
    {
        if (!_holds.TryGetValue(barcode, out var queue))
            _holds[barcode] = queue = new Queue<string>();
        queue.Enqueue(patron);
    }

    public void OnCirculation(object? sender, CirculationEventArgs e)
    {
        if (e.Action != "return") return;

        if (_holds.TryGetValue(e.Item.Barcode, out var queue) &&
            queue.Count > 0)
        {
            string nextPatron = queue.Dequeue();
            HoldReady?.Invoke(this, new HoldReadyEventArgs(nextPatron,
e.Item));
        }
    }
}
```

A `Dictionary<string, Queue<string>>` is a new collection type: a lookup table that maps a key (here, a barcode) to a value (here, a Queue of patron names waiting for that specific item). `_holds["B-001"]` gives you the queue of everyone waiting for barcode B-001, specifically.

A `Queue<string>` is a first-in-first-out collection — the same order people line up at a checkout counter. `.Enqueue(patron)` adds someone to the back of the line; `.Dequeue()` removes and returns the person at the FRONT of the line, in the order they joined.

Notice the method signature of `OnCirculation(object? sender, CirculationEventArgs e)` — it EXACTLY matches the shape required by `EventHandler<CirculationEventArgs>`. This means

we can hand this method DIRECTLY to CheckoutService's event, instead of writing a new lambda:

 FILE: Program.cs

Right after your two existing ItemCirculated subscribers from Step 9.3, in the same spot near the top of the file.

```
var holdWatcher = new HoldWatcher();
checkoutService.ItemCirculated += holdWatcher.OnCirculation;

holdWatcher.HoldReady += (sender, e) =>
    Console.WriteLine($"🔔 Notify {e.Patron}: '{e.Item.Title}' is now
available!");
```

 **Notice the chain that's just been built:** CheckoutService raises ItemCirculated → HoldWatcher listens and, if relevant, raises its OWN event, HoldReady → your console subscriber reacts to THAT. Three completely separate pieces of code, chained together entirely through events, with NONE of them directly calling or even referencing each other's internals. This kind of decoupled chain is exactly what makes large real-world applications maintainable — you can add, remove, or change any one link without touching the others.

 FILE: Program.cs

Anywhere in your menu — add a new option that calls holdWatcher.PlaceHold(barcode, patronName) so you can actually test this end to end.

To exercise this for real, add a menu option that asks for a barcode and a patron name, then calls holdWatcher.PlaceHold(barcode, patronName). Then check out that item, return it, and watch the 🔔 notification fire automatically.

 CHECKPOINT — Milestone 9

- ✓ Placing a hold for a patron on an unavailable item, then returning that item, triggers a console notification, a log file line, AND a hold-ready notification — all from a single GiveBack() call.
- ✓ You can point to the exact line inside CheckoutService that references the console subscriber, the file logger, or the HoldWatcher — and confirm there isn't one. CheckoutService genuinely knows nothing about any of them.
- ✓ You can explain, using the doorbell analogy or your own words, the difference between a delegate and an event.

Milestone 10: Async Programming

What You're Building

Imagine ordering food at a restaurant. After ordering, you don't stand frozen at the counter, staring at the kitchen door, refusing to do anything else until your food arrives — you go sit down, talk to your friends, check your phone. When your food is ready, the staff brings it to you.

Now imagine a badly-run restaurant where, the moment you order, you're physically required to stand still and wait, doing nothing else, until the food arrives — and everyone behind you in line is stuck too, unable to even place THEIR order, because you're blocking the counter.

Without async code, that second restaurant is what your program does any time it waits on something slow, like a network request: it freezes completely, unable to do anything else, even though the waiting itself doesn't require the computer's constant attention. This milestone introduces `async` and `await`, which let your program "go sit down" while waiting, and resume the moment the result is ready — instead of freezing.

 **One thing to get right immediately:** `async/await` is NOT about doing two things at the exact same physical instant. It's about not wasting time standing around doing nothing while you wait for something slow. Keep this distinction in mind throughout this milestone.

Step 10.1 — A Simulated Slow Operation

 **NEW FILE: `AsyncClients.cs`** *(create this file now)*

Create this brand-new file, next to `Events.cs`.

Real slow operations are usually network calls — contacting another library's system, fetching cover art from an external image service. We'll simulate this with `Task.Delay`, which pauses for a given number of milliseconds without freezing anything, the same way a real network call takes time without freezing anything (when written correctly).

```
public class ExternalCatalogClient
{
    public async Task<int> CheckBranchAvailabilityAsync(string branch,
        CancellationToken ct = default)
    {
        await Task.Delay(Random.Shared.Next(150, 400), ct);
        return Random.Shared.Next(0, 4);
    }
}
```

The word `async` in the method signature marks this as an asynchronous method — one that's allowed to pause partway through and resume later, without blocking the calling thread while it waits.

The return type `Task<int>` (instead of plain `int`) signals: "this method's real result isn't ready immediately — it will eventually produce an `int`, wrapped in a `Task` that represents the ongoing work." Think of a `Task` as a claim ticket: you get it back right away, but the actual `int` it represents arrives later.

Inside the method, `await Task.Delay(...)` is where the pause happens. The `await` keyword means: “pause HERE, release control back to whoever called this method (so they can go do other things), and resume running the rest of this method once the delay finishes.”

The `CancellationToken ct = default` parameter is a standard way to let a caller later request “stop this operation early if it's taking too long” — we'll use it properly in Step 10.3.

Step 10.2 — Sequential vs. Concurrent: Seeing the Difference

FILE: Program.cs

Anywhere you want to try this — for example, a new menu option. Important: this code uses `await`, which means the METHOD it lives in must be declared `async`. If you put it directly in your top-level `Program.cs` statements, that's fine — top-level statements in a `Main` file can use `await` directly. If you put it inside one of your own methods (like a new menu handler), that method's signature needs the `async` keyword and probably a `Task` return type — see the warning below.

This is the moment `async` programming actually proves its worth. Let's check four library branches for availability — first one at a time, then all at once — and time both approaches.

```
using System.Diagnostics;

var client = new ExternalCatalogClient();
string[] branches = { "Main", "East", "West", "South" };

// SEQUENTIAL – one branch waits for the previous one to finish
var sw = Stopwatch.StartNew();
foreach (var branch in branches)
    await client.CheckBranchAvailabilityAsync(branch);
Console.WriteLine($"Sequential: {sw.ElapsedMilliseconds}ms");

// CONCURRENT – all four start at once
sw.Restart();
var tasks = branches.Select(b => client.CheckBranchAvailabilityAsync(b));
var results = await Task.WhenAll(tasks);
Console.WriteLine($"Concurrent: {sw.ElapsedMilliseconds}ms");
```

 **If your menu uses a switch statement calling regular (non-async) methods:** you have two options. Easiest: put this whole block directly as a new case in your switch, since top-level `Program.cs` can use `await` freely. Cleaner: write a separate `static async Task CheckBranchesAsync() { ... }` method containing this code, then call it from your switch case with `await CheckBranchesAsync();` — but that only works if the method CONTAINING the switch is also `async`. The reference solution uses this second approach; see the note at the end of this milestone.

In the sequential version, each `await` inside the `foreach` loop fully completes before the next one starts — so the total time is roughly the SUM of all four delays (around 150–400ms each, so somewhere around 1100ms total).

`branches.Select(b => client.CheckBranchAvailabilityAsync(b))` uses `Select` (from Milestone 8's LINQ) to START all four `async` calls at once, WITHOUT awaiting any of them individually yet. Each call begins running immediately, all roughly simultaneously.

`await Task.WhenAll(tasks)` then waits for ALL of those already-running tasks to finish, and collects all their results together into one array. Because they were all running at the same time rather than one after another, the total time is roughly the time of the SLOWEST single one — not the sum of all four.

Run this and read the two numbers in your terminal — you should see the concurrent version finish noticeably faster, even though it's doing the exact same total amount of work.

Step 10.3 — Cancellation

FILE: Program.cs

Right after the sequential/concurrent code from Step 10.2, in the same spot.

Sometimes you don't want to wait indefinitely for a slow operation — you want to give it a time budget, and move on if it doesn't finish in time.

```
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(1));
try
{
    await client.CheckBranchAvailabilityAsync("SlowBranch", cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("🕒 Timed out after 1 second.");
}
```

A `CancellationTokenSource` is an object that can signal “stop now” to any async operation that's listening for it. Passing `TimeSpan.FromSeconds(1)` to its constructor means it will automatically signal cancellation after 1 second has passed, with no manual timer code needed.

Its `.Token` property is what you actually pass into async methods — it's the specific “listening device” those methods check periodically to see if they should give up early.

When cancellation IS triggered, the awaited operation throws an `OperationCanceledException`, which we catch here the same way you've caught exceptions since Milestone 7 — cleanly handling the timeout instead of letting it crash anything.

Step 10.4 — Async Streams: Results That Arrive Over Time

FILE: AsyncClients.cs

Same file as `ExternalCatalogClient`, below it.

Sometimes you don't want all your results at once at the very end — you want to process each one AS it becomes available. Imagine a slow incoming feed of new magazine issues.

```
public static async IEnumerable<Magazine> ImportFeedAsync(int count)
{
    for (int i = 1; i <= count; i++)
    {
        await Task.Delay(150);
        yield return new Magazine("Wired", $"M-F{i:000}", i);
    }
}
```

```
}
```

`IEnumerable<Magazine>` is the return type for a method that produces a SEQUENCE of results over time, asynchronously — rather than one single result, like `Task<int>` did earlier in this milestone.

`yield return` is new syntax: instead of computing everything and returning it all at once with a normal `return`, `yield return` hands back ONE item immediately, then “pauses” this method exactly where it left off, ready to continue from that exact point the next time another item is requested.

FILE: Program.cs

Anywhere you want to consume the feed — for example, a new menu option, right after creating or alongside your catalog setup.

Now consume this stream, processing each magazine the moment it arrives:

```
await foreach (var magazine in ImportFeedAsync(3))
{
    catalog.Add(magazine);
    Console.WriteLine($"Imported: {magazine.Title} Issue
#{magazine.IssueNumber}");
}
```

`await foreach` combines the `foreach` loop you've used since Milestone 5 with the asynchronous waiting from this entire milestone. Each loop iteration awaits the NEXT item from the stream, processes it immediately, then waits for the one after that — rather than waiting for all three magazines to be ready before printing anything.

If you add a `Stopwatch` and print elapsed time inside the loop, you'll see each magazine appear roughly 150ms apart — not all three appearing together after a single 450ms wait. That responsiveness is the entire point of `async` streams.



On structuring your menu around `async` code: Once you have multiple `async` features (branch checks, the magazine feed), it's worth gathering them into their own `async` methods — e.g. `static async Task CheckBranchesAsync()` and `static async Task ImportFeedAsync(...)` — and calling them with `await` from your `switch` statement's case blocks. This keeps the menu-dispatch code looking the same as ever; only the methods that actually do slow work need the `async` keyword.



CHECKPOINT — Milestone 10

- ✓ Your concurrent branch-availability check is measurably faster than the sequential version — you've read both numbers in your terminal and can explain why they differ.
- ✓ A simulated slow operation with a 1-second cancellation budget cancels cleanly, without crashing your program.
- ✓ Imported magazines appear one at a time, with visible time gaps between them, rather than all arriving together at the end.
- ✓ You can explain, using the restaurant analogy or your own words, why `async/await` is about NOT wasting time waiting, rather than about doing things at the exact same instant.

Milestone 11: Reflection-Powered Report Plugins

What You're Building

Think of a restaurant menu. The menu lists every dish available, without you needing to walk into the kitchen and personally check what's being cooked. Reflection is C#'s equivalent: it lets code examine OTHER code while the program is running — what methods exist on a class, what metadata is attached to them — without you having to manually list everything out yourself in advance.

Right now, your six LINQ reports from Milestone 8 are likely wired up with a hand-written switch statement — every time you add a new report, you have to remember to also add a new case. This milestone converts your reports into self-registering plugins: decorate a method with a special marker, and it automatically appears in the menu, with ZERO other code changes required.

Step 11.1 — Define a Custom Attribute

 **NEW FILE: ReportAttribute.cs** *(create this file now)*
Create this brand-new file, next to AsyncClients.cs.

An attribute is a small piece of metadata you can attach to a class, method, or property — extra information ABOUT that piece of code, readable later through reflection.

```
[AttributeUsage(AttributeTargets.Method)]
public class ReportAttribute : Attribute
{
    public string MenuTitle { get; }
    public int Order { get; set; } = 99;

    public ReportAttribute(string menuTitle) => MenuTitle = menuTitle;
}
```

Notice : `Attribute` — inheriting from the built-in `Attribute` class is what makes this usable as an actual attribute (the square-bracket syntax) rather than an ordinary class. This is the same inheritance mechanism from Milestone 5, applied to a built-in base class this time, much like `Exception` was in Milestone 7.

The line directly above the class, `[AttributeUsage(AttributeTargets.Method)]`, is itself an attribute — attached to OUR attribute — restricting where `ReportAttribute` is allowed to be used. `AttributeTargets.Method` means: “this can only be placed on methods, not on classes or properties.” C# will give you a compile error if you try to use it anywhere else.

The property `public int Order { get; set; } = 99;` introduces a small new pattern: a property with a default value (99), and a normal set (not private set like `IsCheckedOut` back in Milestone 4) — meaning whoever USES this attribute can optionally override `Order` if they want their report to appear in a specific position in the menu.

Step 11.2 — Decorate Your Report Methods

 **IN THE PROVIDED SOLUTION** This step asks you to Move your six Milestone 8 reports onto a new instance class — meaning the file you're about to write REPLACES the static

helper methods you wrote in Milestone 8's `Reports.cs`. Open your existing `Reports.cs` and delete its old contents before pasting in the version below; this is a rewrite of that file, not an addition to it.

FILE: Reports.cs

Open your existing `Reports.cs` from Milestone 8 and replace its entire contents with the version below.

Move your six reports into a dedicated class, and decorate each method with your new attribute:

```
public class Reports
{
    private readonly IRepository<LibraryItem> _catalog;

    public Reports(IRepository<LibraryItem> catalog) => _catalog =
catalog;

    [Report("Available items", Order = 1)]
    public string AvailableItems()
    {
        var items = _catalog.GetAll().Where(i => !
i.IsCheckedOut).OrderBy(i => i.Title);
        return string.Join("\n", items.Select(i => i.Title));
    }

    [Report("Overdue fees", Order = 4)]
    public string OverdueFees()
    {
        var overdue = _catalog.GetAll().Where(i => i.IsOverdue).ToList();
        return $"{overdue.Count} overdue, {overdue.Sum(i =>
i.CurrentFee):C} owed";
    }

    // ... your other four reports from Milestone 8, each with its own
[Report] attribute
}
```

The square-bracket syntax `[Report("Available items", Order = 1)]` directly above a method attaches an instance of your `ReportAttribute` to it. The text in quotes is passed to the constructor (becoming `MenuTitle`); `Order = 1` is a named-property assignment, setting that specific property on the attribute instance after construction. This is special syntax **ONLY** available when applying attributes — you won't see this exact pattern anywhere else in C#.

Notice each report method now **RETURNS** a string (the formatted report text) instead of printing directly with `Console.WriteLine`. This is a deliberate, useful change: it separates "what the report says" from "where it gets displayed," which matters because in Milestone 12 we'll display these same reports through a Web API instead of the console — and a Web API can't "print" to anyone's screen.

 **IN THE PROVIDED SOLUTION** Your sixth report — Search — takes parameters (a title fragment and a type filter), but the discovery engine you're about to write in Step 11.3 only knows how to invoke methods with zero parameters. Leave Search OFF the [Report] attribute entirely; keep it as a plain public method on the Reports class (or call it some other way from your menu, passing in the title/type the user typed). The other five reports are exactly the kind of zero-argument, string-returning methods the attribute is designed for.

Step 11.3 — Build the Discovery Engine

 **NEW FILE: ReportRunner.cs** *(create this file now)*
Create this brand-new file, next to ReportAttribute.cs.

Now the genuinely new part: code that scans the Reports class at runtime, finds every method with our attribute attached, and builds a menu automatically.

```
using System.Reflection;

public static class ReportRunner
{
    public record DiscoveredReport(string MenuTitle, int Order,
    Func<string> Run);

    public static List<DiscoveredReport> Discover(object reportsInstance)
    {
        return reportsInstance.GetType()
            .GetMethods(BindingFlags.Public | BindingFlags.Instance)
            .Select(m => (Method: m, Attr:
m.GetCustomAttribute<ReportAttribute>()))
            .Where(x => x.Attr is not null)
            .OrderBy(x => x.Attr!.Order)
            .Select(x => new DiscoveredReport(
                x.Attr!.MenuTitle,
                x.Attr!.Order,
                () => (string)x.Method.Invoke(reportsInstance, null!))
            ).ToList();
    }
}
```

This is dense — let's go through it one piece at a time, since it's genuinely the most advanced code in this entire guide.

`reportsInstance.GetType()` is reflection's entry point: given any actual object, `.GetType()` returns a `Type` object representing its CLASS — here, the `Reports` class — which you can then inspect.

`.GetMethods(BindingFlags.Public | BindingFlags.Instance)` asks that `Type` object: “list every public, non-static method you have.” This returns information ABOUT each method — its name, its parameters, its attributes — not the ability to run it yet.

`m.GetCustomAttribute<ReportAttribute>()` checks a specific method `m` for our custom attribute. If the method WASN'T decorated with `[Report(...)]`, this returns null. If it WAS, it returns the actual attribute instance, letting us read `MenuTitle` and `Order` off of it.

The whole chain — `Select`, then `Where(x => x.Attr is not null)`, then `OrderBy` — is LINQ from Milestone 8, applied here to a collection of `METHOD` descriptions rather than `LibraryItems`. This is exactly the payoff of having learned LINQ already: reflection naturally produces collections, and LINQ is the natural tool for filtering and sorting them.

The final piece, `() => (string)x.Method.Invoke(reportsInstance, null)!`, builds a NEW lambda that, when eventually called, will use reflection to ACTUALLY RUN the discovered method. `.Invoke(reportsInstance, null)` means “call this method, on this specific object, with no arguments,” and returns whatever that method returns — here cast to string, since we know every report method returns one.

 **Why wrap it in a lambda instead of calling `Invoke` immediately?** Because `Discover()` is meant to build a MENU — a list of available reports the user can choose from — not to run all six reports immediately, every single time the menu is displayed. Wrapping the actual invocation in a `Func<string>` lambda lets us defer running it until the specific moment the user picks that particular report.

Step 11.4 — Build the Menu From Discovered Reports

FILE: Program.cs

Replace your hand-written reports switch statement from Milestone 8 with the code below — this goes wherever you currently show the reports menu (likely inside a case in your main switch, or in a small submenu method).

```
var reports = new Reports(catalog);
var discovered = ReportRunner.Discover(reports);

for (int i = 0; i < discovered.Count; i++)
    Console.WriteLine($"{i + 1}. {discovered[i].MenuTitle}");

Console.Write("Pick a report: ");
if (int.TryParse(Console.ReadLine(), out int pick) && pick >= 1 && pick <=
discovered.Count)
    Console.WriteLine(discovered[pick - 1].Run());
```

Now, to prove this genuinely works the way it claims to: add a brand-new method to your `Reports` class, decorate it with `[Report("My New Report")]`, and run your program again — without touching the menu-building code above AT ALL. Your new report appears in the numbered list automatically.

CHECKPOINT — Milestone 11

- ✓ Your reports menu is generated entirely by `ReportRunner.Discover` — there is no hand-written switch statement listing each report.
- ✓ Adding a brand-new `[Report]`-decorated method makes it appear in the menu without modifying any other code.
- ✓ You can explain, using the restaurant menu analogy or your own words, what reflection lets code do that it couldn't do otherwise.
- ✓ You understand why each report method now returns a string instead of printing directly — and can anticipate why that decision will matter in Milestone 12.

Milestone 12 (Capstone): The DeweyLib Web API

What You're Building

Think about a restaurant menu again, but this time think of the menu itself as a **CONTRACT** between you and the kitchen. You don't walk into the kitchen and cook your own food — you tell a waiter what you want from the menu, and the kitchen sends back a finished plate.

A Web API works exactly the same way. Other programs — a mobile app, a website, another company's server — don't reach directly into your code. They ask for something specific over the internet ("give me the list of books") and your API sends back an answer, the same way a kitchen sends back a plate.

Here is the genuinely exciting part: everything you've built across the previous eleven milestones — the Book/Dvd/Magazine hierarchy, the repository, the checkout service, the reports — already exists and already works. This milestone doesn't replace any of that. It adds a "waiter" on top: a way for other programs to ask for it over a web address, instead of through a direct method call inside the same program.

Step 12.1 — Create the Web API Project

 **NEW FILE: (new folder, alongside your existing project)** *(create this file now)*

This step creates TWO brand-new project folders as siblings of your existing DeweyLib folder, not inside it. By the end of this step you'll have three project folders sitting side by side:

DeweyLib.Core, DeweyLib.Console (your renamed existing project), and DeweyLib.Api.

First, let's reorganize slightly. Move your domain classes — `LibraryItem` and its subclasses, `IRepository`, `CheckoutService`, the exceptions, `Reports`, `ReportRunner` — into their own separate project, a class library, so both a console app AND a web API can reference the same code without duplicating it.

```
dotnet new classlib -n DeweyLib.Core
dotnet new webapi -n DeweyLib.Api --use-minimal-apis
cd DeweyLib.Api
dotnet add reference ../DeweyLib.Core/DeweyLib.Core.csproj
```

`dotnet new classlib` creates a class library — a project that doesn't run on its own, but exists purely to be **REFERENCED** by other projects that need its code.

`dotnet new webapi -n DeweyLib.Api --use-minimal-apis` creates a new ASP.NET Core Web API project, using the Minimal API style — endpoints defined directly and concisely, without separate controller classes.

`dotnet add reference ../DeweyLib.Core/DeweyLib.Core.csproj` tells the Api project "you are allowed to use classes defined in DeweyLib.Core." Without this, the Api project would have no way to see your `LibraryItem`, `CheckoutService`, and other classes.

 **IN THE PROVIDED SOLUTION** This step's instructions are slightly compressed; here is the exact file move the reference solution performs, since "move your domain classes" is easy to get wrong:

- Move these files, UNCHANGED, from your old single-project folder into the new DeweyLib.Core folder: LibraryItem.cs, Repository.cs, Exceptions.cs, CheckoutService.cs, LibraryStorage.cs, Events.cs, AsyncClients.cs, ReportAttribute.cs, ReportRunner.cs, Reports.cs.
- Your old Program.cs (the console menu) does NOT move into DeweyLib.Core. Either rename your old project folder to DeweyLib.Console and keep Program.cs there, adding `dotnet add reference ../DeweyLib.Core/DeweyLib.Core.csproj` to it too — or, if you'd rather not keep maintaining the console app, you can set it aside and focus only on the Api project from here. Either way, the API needs its OWN Program.cs (Steps 12.3–12.5 below), which is a completely different file from your console app's Program.cs.
- Delete the placeholder Class1.cs file that `dotnet new classlib` generates automatically inside DeweyLib.Core — it's just a sample stub.

Move all your domain class files (Items, Repositories, CheckoutService, Exceptions, Reporting) into the DeweyLib.Core project. They should compile completely unchanged — this is the entire architectural payoff of the layering you built starting back in Milestone 6.

Step 12.2 — Understanding ASP.NET Core's Request Pipeline

FILE: (reading only)

No code yet in this step.

Before writing endpoint code, it helps to understand the shape of how a Web API processes a request. Every incoming HTTP request flows through a PIPELINE of middleware — a sequence of processing steps — before reaching the code that actually handles it, and the response flows back out through that same pipeline.

- Middleware can handle exceptions (turning a thrown exception into an HTTP error response)
- Middleware can route the request (matching a URL like `/api/items/B-001` to the right piece of code)
- Eventually, your specific endpoint code runs and produces a response

Dependency Injection (DI) is the other crucial ASP.NET Core concept: instead of your endpoint code creating a `CheckoutService` itself, you register it ONCE at startup, and ASP.NET Core automatically supplies a ready-made instance to any endpoint that asks for one, by simply listing it as a parameter.

Step 12.3 — Register Your Services

FILE: DeweyLib.Api/Program.cs

This is the Web API project's OWN Program.cs — created automatically by `dotnet new webapi`. It is a completely different file from your console app's Program.cs, even though they share the same name. Open the one inside the DeweyLib.Api folder and replace its starter content with the code below.

Open Program.cs in the Api project and add this near the top:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<IRepository<LibraryItem>>(_ =>
```

```

    var repo = new InMemoryRepository<LibraryItem>();
    repo.Add(new Book("Dune", "B-001", "Frank Herbert"));
    repo.Add(new Dvd("Inception", "D-101"));
    return repo;
});

builder.Services.AddSingleton<CheckoutService>();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

```

`builder.Services.AddSingleton<IRepository<LibraryItem>>(...)` registers a single, shared instance of your repository, created using the lambda provided, which any part of the application can later request. `AddSingleton` specifically means “create this ONE time, and hand the same instance to everyone who asks for it” — appropriate here, since we want one shared in-memory catalog for the whole running application.

Notice we registered it as `IRepository<LibraryItem>` — the interface — not `InMemoryRepository<LibraryItem>` directly. This is the exact same “declare against the interface” principle from Milestone 6, now applied to dependency injection: any endpoint that asks for an `IRepository<LibraryItem>` will receive this `InMemoryRepository` instance, but doesn't need to know that specific detail.

`builder.Services.AddSwaggerGen()` registers Swagger — a tool that automatically generates interactive, browsable documentation for your entire API, letting you test every endpoint directly from a web browser without writing any separate testing tool.

 **IN THE PROVIDED SOLUTION** As of late 2025, `Swashbuckle.AspNetCore` (the package that provides `AddSwaggerGen/UseSwagger/UseSwaggerUI`) released a major version 10 specifically for .NET 10 compatibility. If `dotnet new webapi` didn't already add it for you, add it with:

```
dotnet add package Swashbuckle.AspNetCore --version 10.2.2
```

(or any current 10.x release — check nuget.org for the latest if this one has been superseded by the time you read this).

Step 12.4 — Add Exception-Handling Middleware

 **NEW FILE: `DeweyLib.Api/LibraryExceptionHandler.cs`** *(create this file now)*
Create this brand-new file inside the `DeweyLib.Api` project folder (not `Core`, not `Console`) — it's specific to how THIS project translates exceptions into HTTP responses.

Recall from Milestone 7 that `CheckoutService` **THROWS** exceptions rather than returning `true/false`. We need middleware that catches those specific exceptions and turns them into proper HTTP error responses.

```

using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;

public class LibraryExceptionHandler : IExceptionHandler

```

```

{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext ctx, Exception ex, CancellationToken ct)
    {
        var (status, title) = ex switch
        {
            ItemNotFoundException => (404, "Item Not Found"),
            ItemUnavailableException => (409, "Item Unavailable"),
            _ => (500, "Internal Server Error"),
        };

        ctx.Response.StatusCode = status;
        await ctx.Response.WriteAsJsonAsync(new ProblemDetails
        {
            Status = status, Title = title, Detail = ex.Message
        }, ct);

        return true;
    }
}

```

This class implements the built-in `ExceptionHandler` interface (the same "implements" relationship from Milestone 6's `IRepository`) — ASP.NET Core itself defines this contract, and any class fulfilling it can be plugged into the pipeline to handle exceptions globally, in one place, instead of wrapping every single endpoint in its own `try/catch`.

The switch expression here is the exact same pattern from Milestone 7's JSON storage code (i switch { ... }) — just matching on `EXCEPTION TYPES` this time, to decide which HTTP status code and title to use. 404 means "not found"; 409 means "conflict" (the request was understood, but couldn't be completed given the current state — perfect for "already checked out"); 500 means "something unexpected went wrong on our end."

`ProblemDetails` is a standard, built-in shape for describing HTTP errors as JSON — using it means any client talking to your API gets errors in a predictable, well-documented format, rather than your own custom one-off error shape.

FILE: DeweyLib.Api/Program.cs

Back in the `Api` project's `Program.cs`. Add these two lines alongside your other `builder.Services...` lines from Step 12.3 (before `var app = builder.Build();`), and the two `app.Use...` lines directly after `var app = builder.Build();`.

Register this handler alongside your other services, and activate it in the pipeline:

```

builder.Services.AddExceptionHandler<LibraryExceptionHandler>();
builder.Services.AddProblemDetails();

// ... after var app = builder.Build(); ...
app.UseExceptionHandler();
app.UseSwagger();
app.UseSwaggerUI();

```

Step 12.5 — Define Your Endpoints

FILE: DeweyLib.Api/Program.cs

Same file, directly below `app.UseSwaggerUI();` and before the final `app.Run();` line.

Now, the part this entire milestone has been building toward: actual endpoints, defined directly and concisely.

```
var items = app.MapGroup("/api/items");

items.MapGet("/", (IRepository<LibraryItem> repo) =>
    Results.Ok(repo.GetAll().Select(i => new { i.Title, i.Barcode,
i.IsCheckedOut })));

items.MapGet("/{barcode}", (string barcode, IRepository<LibraryItem> repo)
=>
{
    var item = repo.Find(i => i.Barcode == barcode);
    return item is null ? Results.NotFound() : Results.Ok(item);
});

items.MapPost("/{barcode}/checkout", (string barcode, CheckoutService svc)
=>
{
    var item = svc.Borrow(barcode); // throws → LibraryExceptionHandler
catches it
    return Results.Ok(new { item.Title, item.DueDate });
});

app.Run();
```

`app.MapGroup("/api/items")` creates a group of related endpoints all sharing the same URL prefix — every endpoint defined on `items` below will live somewhere under `/api/items/...`

Look closely at the parameters in each lambda: `(IRepository<LibraryItem> repo) => ...` and `(string barcode, CheckoutService svc) => ...`. This is dependency injection in action: `barcode` comes from the URL itself (matching the `{barcode}` placeholder in the route), while `repo` and `svc` are AUTOMATICALLY supplied by ASP.NET Core, using the services you registered in Step 12.3. You never write code that manually creates a `CheckoutService` here — the framework hands you a ready-made one.

Notice the checkout endpoint: `svc.Borrow(barcode);` is the EXACT same method from `CheckoutService`, written back in Milestone 7, completely unchanged. If the barcode doesn't exist or the item is already checked out, it throws — exactly as before — and now `LibraryExceptionHandler` (Step 12.4) catches that exception and turns it into the correct HTTP response automatically.

 **Stop and appreciate this:** The exact same `ItemNotFoundException` that printed a friendly console message back in Milestone 7 now becomes an HTTP 404 response with a JSON body, in a completely different kind of application, using code in `CheckoutService` that has not changed by a single character. This is the entire payoff of the layered architecture

you've been building since Milestone 6: business logic that doesn't know or care what kind of user interface is sitting on top of it.

 **IN THE PROVIDED SOLUTION** The reference solution also adds a matching `items.MapPost("/{barcode}/return", ...)` endpoint calling `svc.GiveBack(barcode)`, and a `app.MapGet("/api/reports", ...)` endpoint that runs `ReportRunner.Discover` (Milestone 11) against the repository and returns every discovered report's title and rendered text as JSON — proving the reflection-based reports work over HTTP exactly as they did on the console, with no changes. Both are optional extensions of the lab text's three endpoints, included to round out the API; add them if you'd like the same coverage.

Step 12.6 — Run It and Test With Swagger

FILE: (terminal)

Run this from inside the `DeweyLib.Api` folder specifically — not the solution root, not `DeweyLib.Core`.

```
dotnet run
```

Open the URL shown in your terminal, adding `/swagger` to the end (for example, `https://localhost:5001/swagger`). You'll see an interactive page listing every endpoint you defined. Click on `GET /api/items`, then "Try it out", then "Execute" — you'll see your actual catalog data returned as JSON, directly in the browser.

Try checking out the same barcode twice using the checkout endpoint. The first attempt should succeed with a 200 status. The second should fail with a 409 Conflict — and if you look at the response body, you'll see a `ProblemDetails` object whose Detail message comes directly from the `ItemUnavailableException` your `CheckoutService` threw.

 **If you see: Internal Server Error / 500 on every request** → Almost always means the exception handler middleware wasn't registered correctly, or a required service wasn't registered with `builder.Services` before calling `builder.Build()`. Double check Step 12.3 and 12.4 ran in the right order, before `var app = builder.Build();`.

 **If you see: Swagger page doesn't load** → Usually means `app.UseSwagger();` or `app.UseSwaggerUI();` is missing, or was placed in the wrong order relative to other middleware. They should generally come early in the pipeline, right after `app.UseExceptionHandler();`.

CHECKPOINT — Milestone 12 — The Complete Application

- ✓ Every endpoint responds correctly when tested through the Swagger UI.
- ✓ A double-checkout attempt returns HTTP 409 with a `ProblemDetails` body whose Detail message comes from your own `ItemUnavailableException`.
- ✓ A request for a barcode that doesn't exist returns HTTP 404, again using your own `ItemNotFoundException`'s message.
- ✓ You can trace, out loud, the complete path a request takes: HTTP request → routing → your endpoint lambda → `CheckoutService` → the `LibraryItem` object itself → back out through the same path, ending as an HTTP response.

You're Finished — What You Actually Built

Take a moment to recognize what just happened. You started with one line — `Console.WriteLine("Hello, World!");` — and ended with a layered application containing:

- A polymorphic object hierarchy (Book, Dvd, Magazine, all sharing `LibraryItem`'s behavior automatically)
- A generic, swappable storage layer (`IRepository<T>`, usable for any data type)
- Proper error handling with custom, information-rich exceptions
- Data that survives a restart, saved and loaded as JSON
- Powerful, readable queries and reports built with LINQ
- A fully decoupled notification system built on events
- Responsive, non-blocking code using `async/await`, including live streaming results
- A self-registering plugin system built with reflection
- A real, working REST API that any other program on the internet could talk to

Every single one of these ideas was introduced gradually, on top of something you already understood from the milestone before it. There was no point where you were asked to absorb something with no foundation underneath it. That's not an accident — it's also exactly how you should approach learning anything new in software: find the smallest version of the idea, get it working, and then layer the next idea carefully on top.

 **Congratulations — you built DeweyLib.** Keep this document, and keep your code. The patterns here — encapsulation, polymorphism, programming against interfaces, services that throw while UIs catch, decoupling through events, and layering a UI on top of a UI-agnostic core — are not specific to libraries or to this exercise. You will use every one of them again, in every serious application you ever build.

APPENDIX

Deviations Log

Every place the provided reference solution differs from this guide's bare instructions, gathered in one place.

This guide is written to be followed exactly, step by step, and in almost every milestone the code you'll find in the reference solution matches what's printed here character-for-character. A handful of spots needed something the lab text doesn't spell out, or contained an instruction that — followed literally — would quietly break a later checkpoint. Those spots are flagged inline, right where they occur, in purple boxes marked “IN THE PROVIDED SOLUTION.” This appendix collects all of them in one place, so you can check your own work against the complete list, or skim it before you start if you'd rather know what's coming.

Milestone 5 — File organization begins

Not a code change, but a structural one: starting at M5, classes move out of `Program.cs` and into their own files (`LibraryItem.cs`, then `Repository.cs`, `Exceptions.cs`, and so on through the rest of the guide). The lab text doesn't dictate this explicitly; it's standard C# practice once you have more than one or two related classes, and it's the reason every step from M5 onward opens with a 📁 FILE box telling you exactly where new code belongs.

Milestone 7 — `Load()` must restore checkout state

The lab text's `LibraryStorage.Load` rebuilds each item from its saved record but never calls `Checkout()` or restores its `DueDate`. Followed literally, every item comes back from a restart marked `Available` — which directly contradicts this milestone's own checkpoint (“still checked out after a restart”). The reference solution's `Load` adds: when a record's `IsCheckedOut` is true, call `item.Checkout()` and then use `BackdateForDemo` (from Milestone 5) to set `DueDate` back to the exact value that was saved, rather than the fresh one `Checkout()` would otherwise compute.

Milestone 8 — `CurrentFee` is used but never defined

Report 4's `overdue.Sum(i => i.CurrentFee)` calls a property the lab text never shows you writing. Add it to `LibraryItem.cs`, next to `DaysLate` (Milestone 5):

- `public decimal CurrentFee => DaysLate * DailyLateFee;`

Milestone 11 — `Reports.cs` is rewritten, not extended

Step 11.2 says to “move” your six Milestone 8 reports onto a new instance class — in practice this means deleting the static helper methods you wrote for `Reports.cs` in Milestone 8 and replacing the whole file with the attribute-decorated instance version. It's easy to read “move” as “add alongside” and end up with duplicate, conflicting report logic in the same file.

Separately: `Search` (Report 6) takes parameters, but the reflection-based `ReportRunner.Discover` only knows how to invoke zero-argument methods. `Search` is deliberately left off the `[Report]` attribute and stays a plain method, called directly from the menu with whatever the user typed.

Milestone 12 — exact project layout and two extra endpoints

The lab text's instruction to "move your domain classes" into DeweyLib.Core and create DeweyLib.Api is correct in spirit but light on mechanics. The reference solution:

- Moves LibraryItem.cs, Repository.cs, Exceptions.cs, CheckoutService.cs, LibraryStorage.cs, Events.cs, AsyncClients.cs, ReportAttribute.cs, ReportRunner.cs, and Reports.cs — unchanged — into DeweyLib.Core.
- Renames the original single-project folder to DeweyLib.Console and keeps Program.cs (the console menu) there, referencing DeweyLib.Core, so the console app from Milestones 1–11 keeps working alongside the new API rather than being discarded.
- Adds a POST /api/items/{barcode}/return endpoint (a mirror of the checkout endpoint, calling `svc.GiveBack`) and a GET /api/reports endpoint that runs the Milestone 11 discovery engine over HTTP. Neither is in the lab text's three sample endpoints; both are natural extensions that exercise code you've already written.

Also: Swashbuckle.AspNetCore, the package behind Swagger, released a new major version (10.x) specifically for .NET 10 compatibility. If your project template doesn't pull it in automatically, install it explicitly with `dotnet add package Swashbuckle.AspNetCore --version 10.2.2` (or whatever current 10.x release is newest when you read this).

A note on what didn't change

Every piece of domain logic — the LibraryItem hierarchy, the repository, the exceptions, the checkout service's rules, the events, the async clients, the reflection engine — matches the lab text exactly, line for line, in the reference solution. The deviations above are entirely about filling small, specific gaps (an undefined property, a Load that needs one more block to satisfy its own checkpoint) and about mechanics the lab text gestures at without fully spelling out (which exact files move where in Milestone 12). None of them change what any milestone teaches or how its code behaves once corrected; they exist so that following this guide produces a project that actually passes its own checkpoints, end to end.